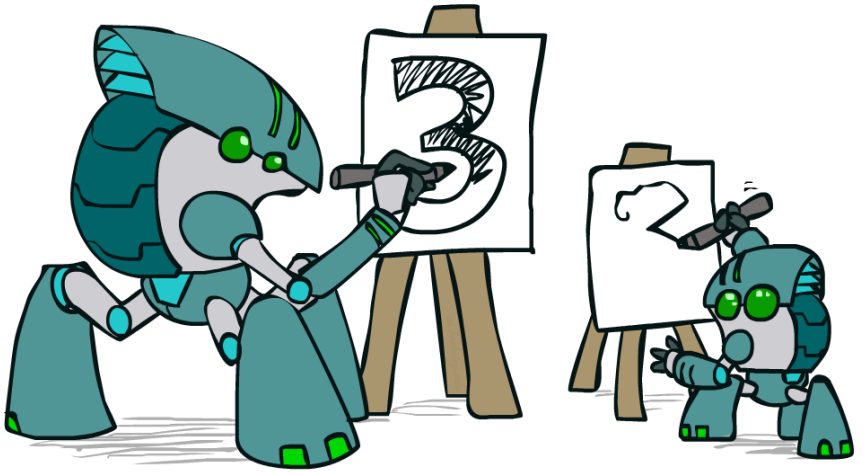
Introduction to Artificial Intelligence

Lecture 7: Machine learning and neural networks

Prof. Gilles Louppe
g.louppe@uliege.be

LIÈGE université

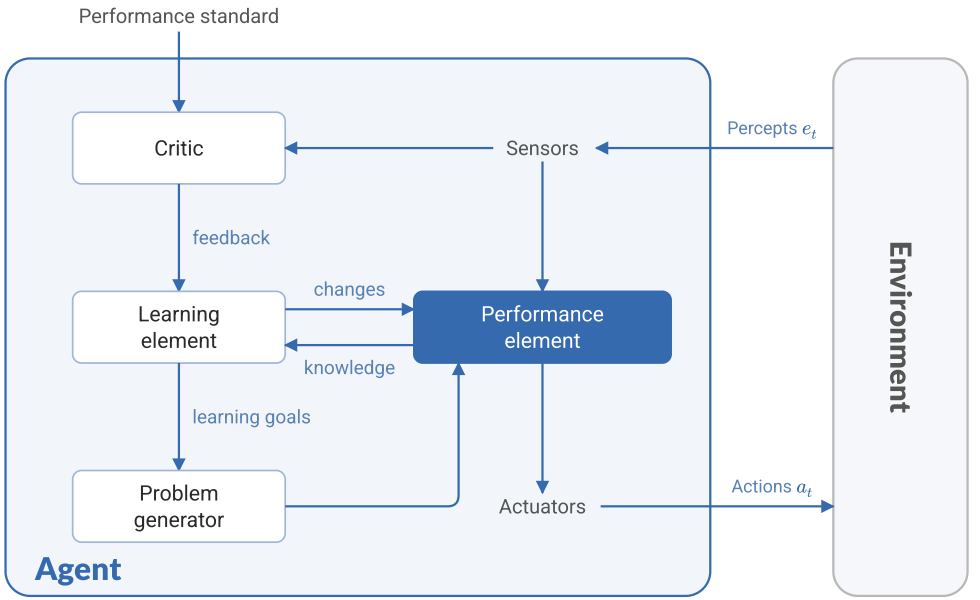
Today



Learning from data is a key component of artificial intelligence. In this lecture, we will introduce the principles of:

- Machine learning
- Neural networks

Learning agents



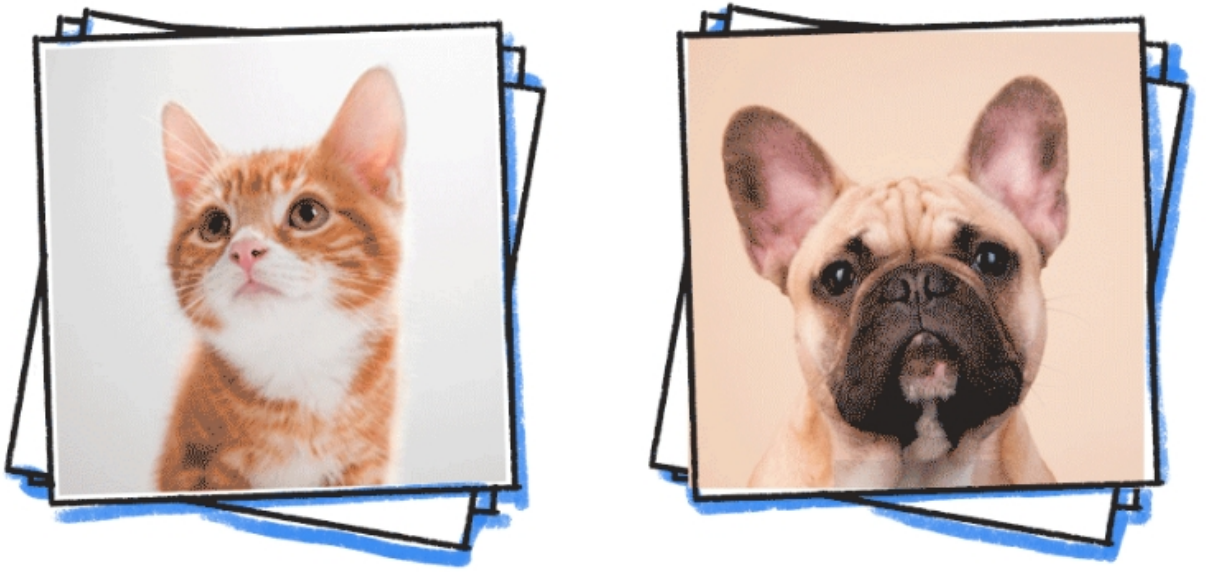
What if the environment is **unknown**, or too hard to program by hand?

A learning agent improves its performance element with experience (Lecture 1). Here, the performance element is a function with parameters θ:

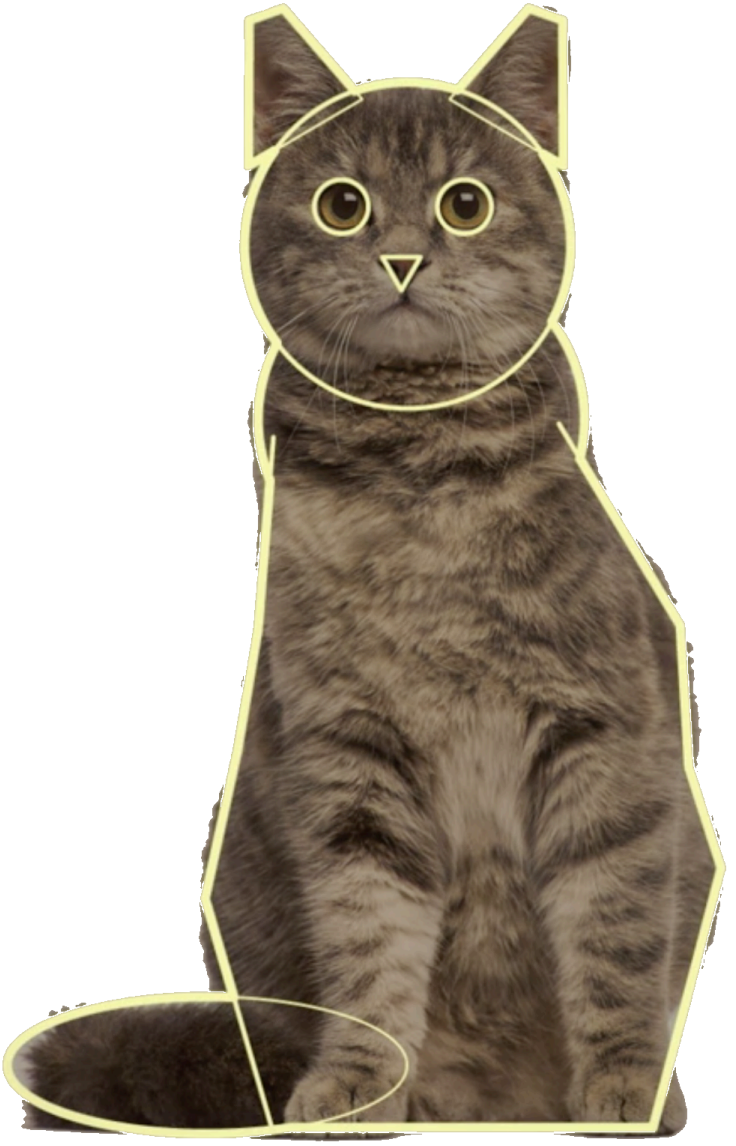
- a policy $a_t = \pi_\theta(e_t)$ (Lecture 1),
- a heuristic $h(n)$ (Lecture 2) or an evaluation function $\mathbf{eval}(s)$ (Lecture 3),
- a model $\mathbf{P}_\theta(X)$ (Lecture 5), such as the transition and sensor models of Lecture 6.

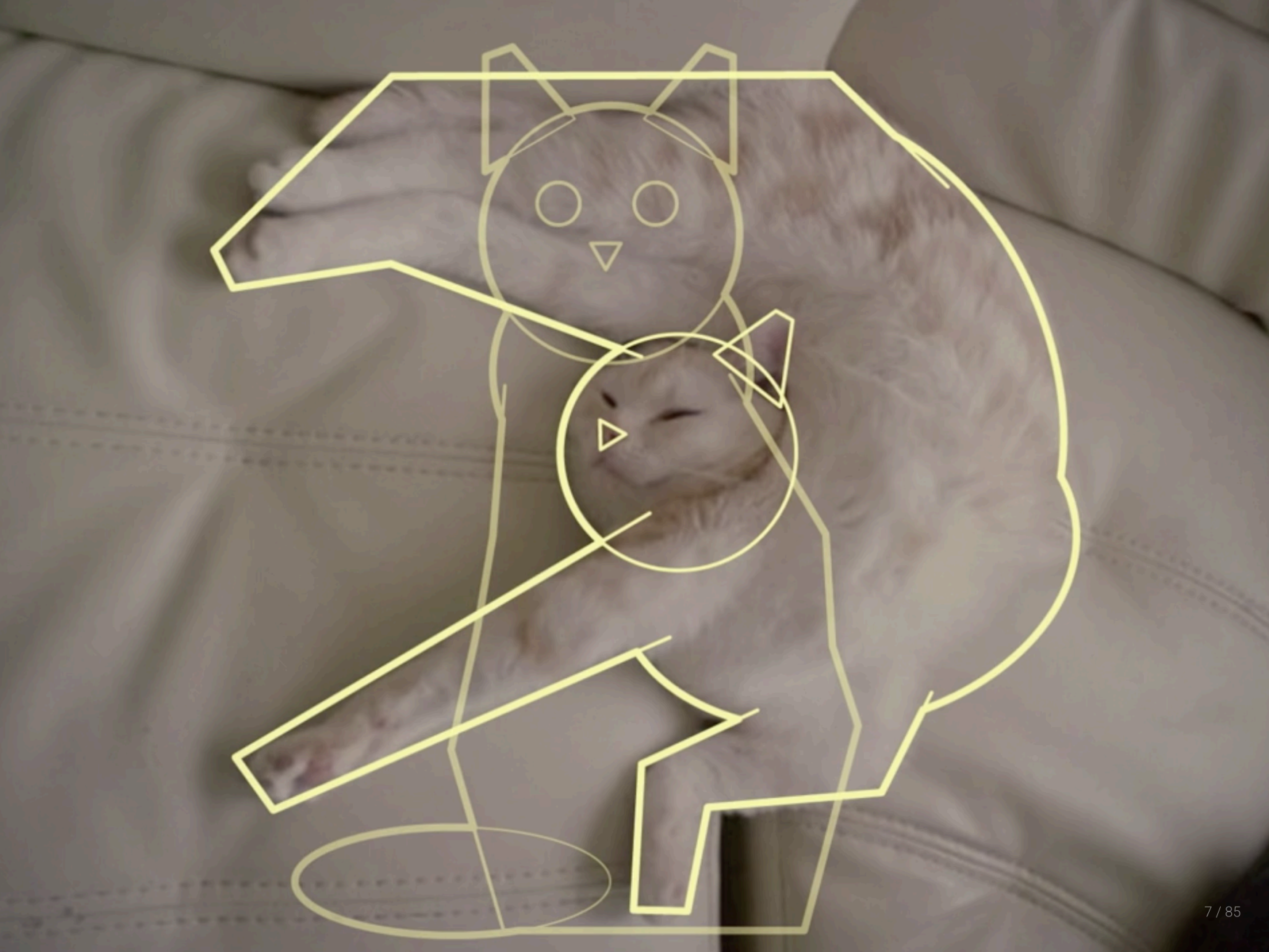
The learning element adjusts θ from the feedback.

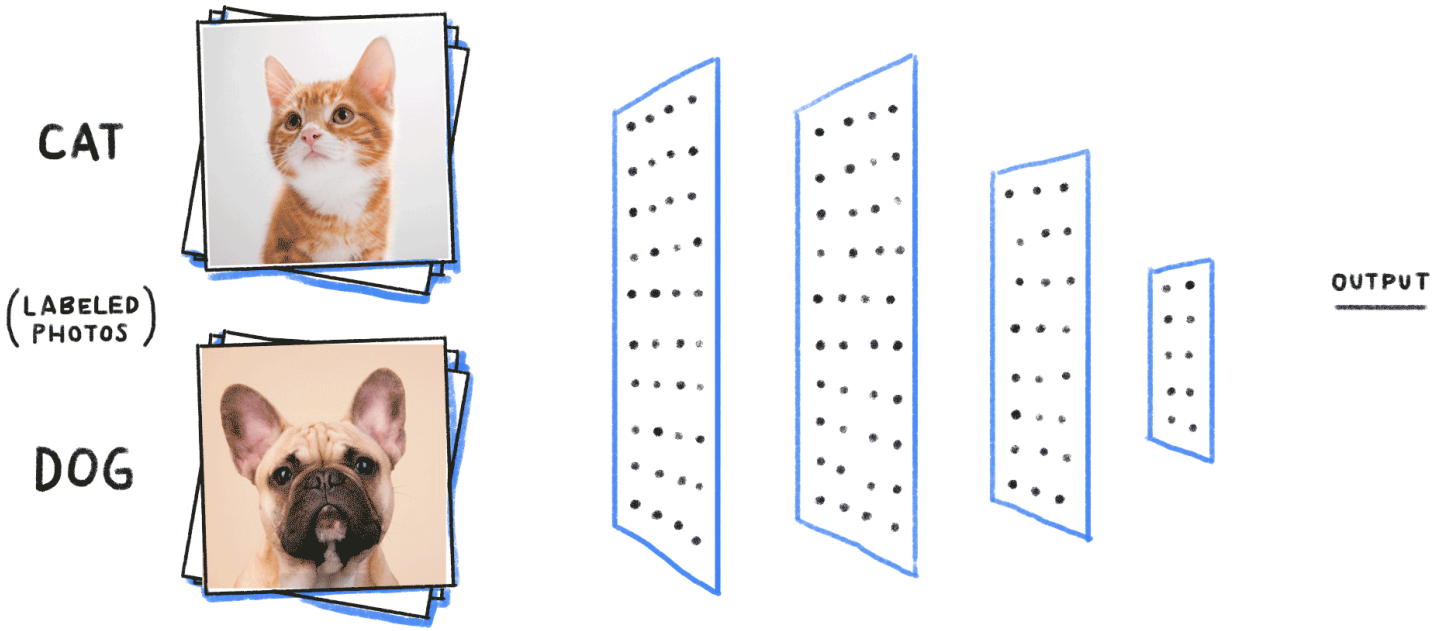
Machine learning



How would you write a computer program that recognizes cats from dogs?







The deep learning approach: train a highly parameterized model to produce correct outputs (e.g., class labels) from inputs (e.g., images).

Supervised learning

Let $\mathbf{d}$ be a dataset of N input-output pairs, drawn i.i.d. from $p(\mathbf{x}, y)$,

$$\mathbf{d} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), ..., (\mathbf{x}_N, y_N)\},$$

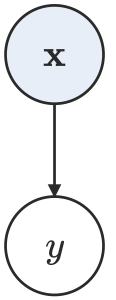
with inputs $\mathbf{x}_i \in \mathbb{R}^d$ and outputs $y_i \in \mathcal{Y}$.

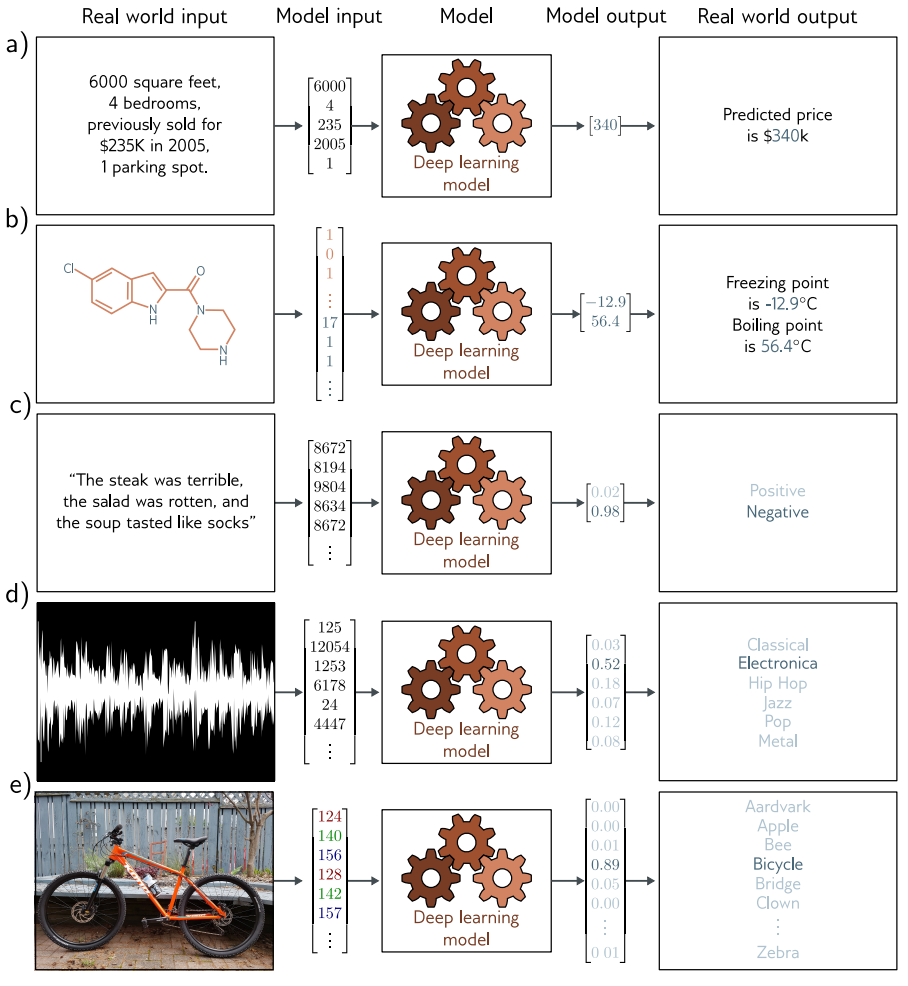
From this data, we want to identify a probabilistic model

$$p_\theta(y \mid \mathbf{x})$$

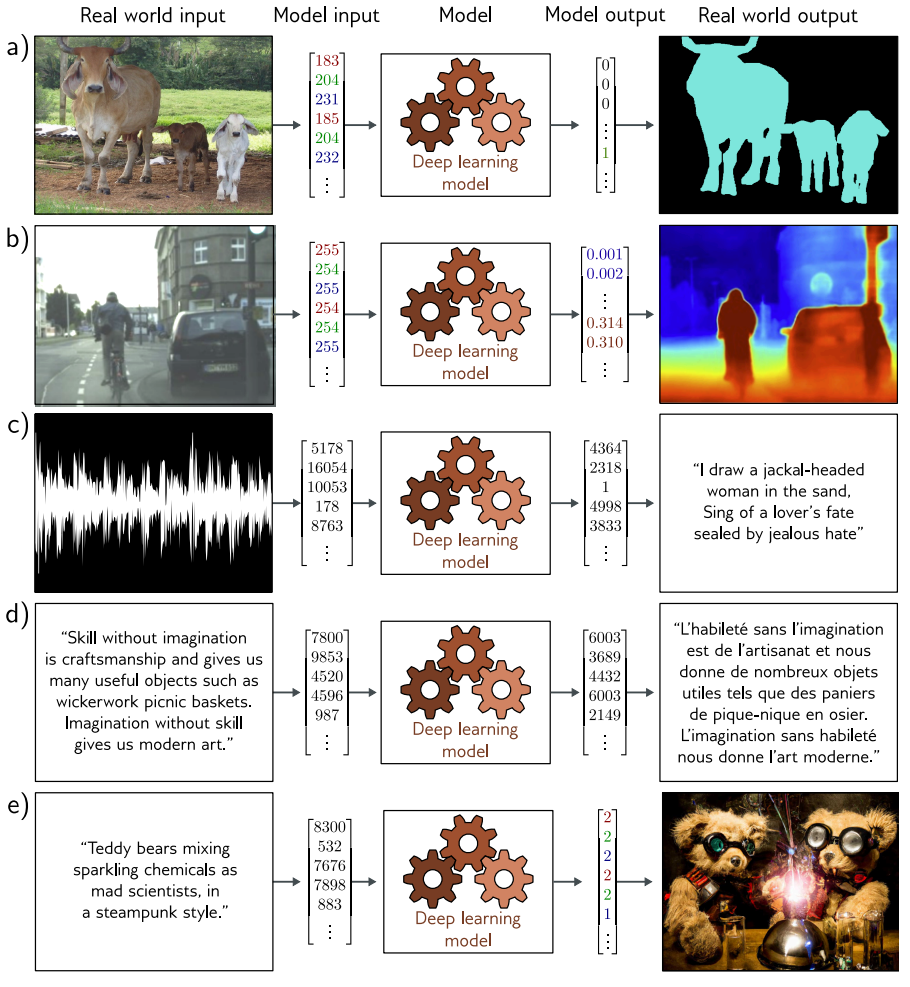
that best explains the data.

As in Lectures 4 to 6, the model is a density p for continuous y and a probability P for discrete y.

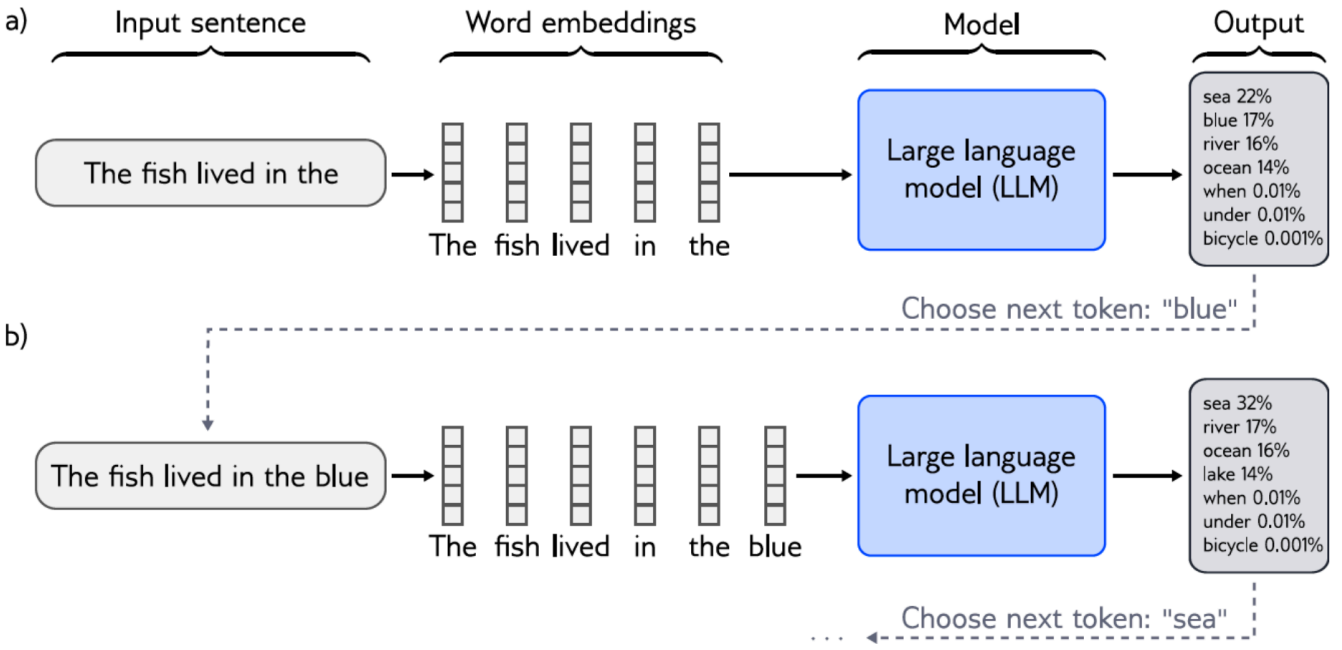




Regression ($y \in \mathbb{R}$) and classification ($y \in \{0, 1, ..., C-1\}$) problems, with inputs and outputs written as numbers.



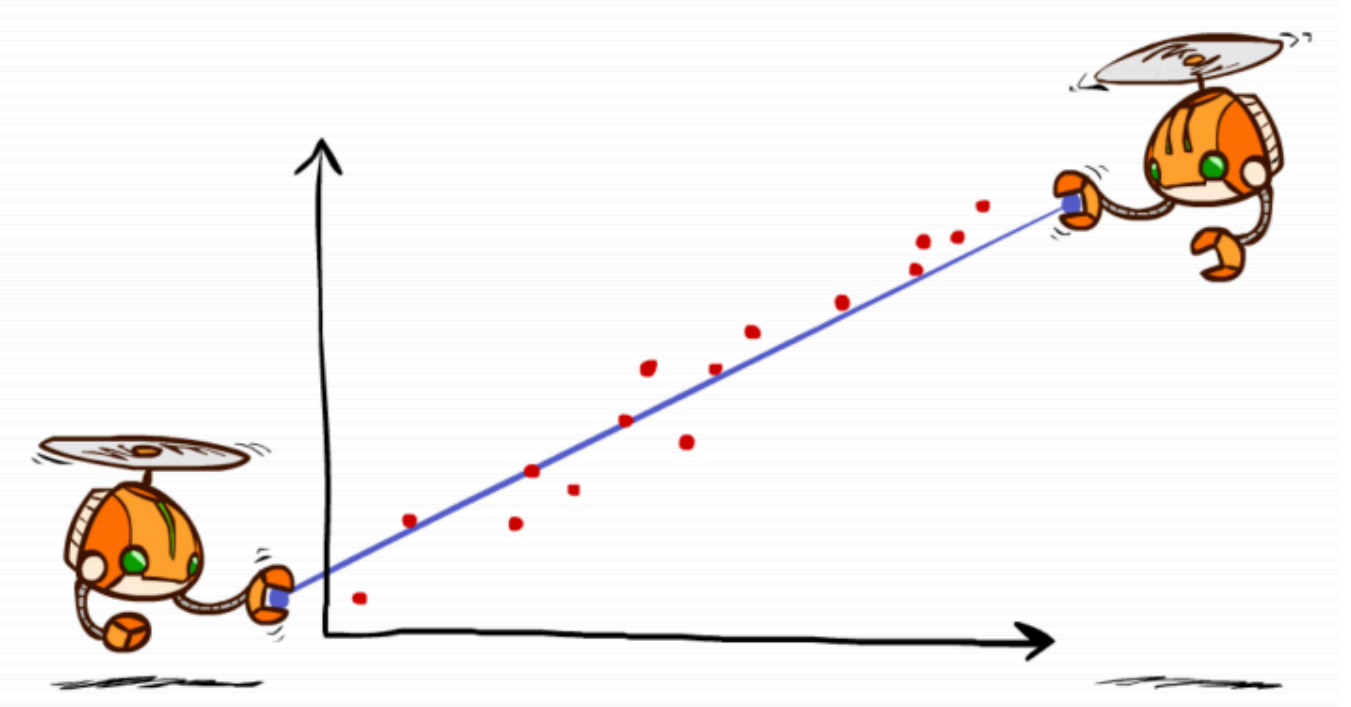
Supervised learning with structured outputs ($y \in \mathcal{Y}$).

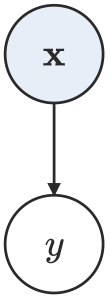


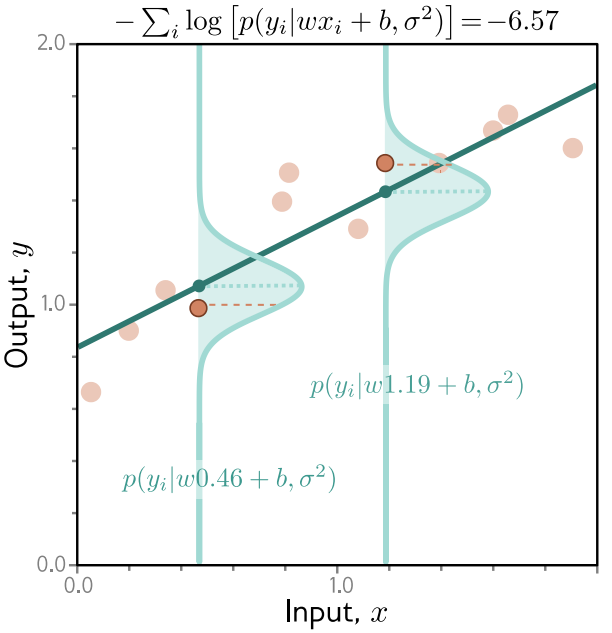
Language models predict the next token, a classification over the vocabulary.

Linear regression

Let us first assume that $y \in \mathbb{R}$.







Linear regression models $p(y \mid \mathbf{x})$ as a linear Gaussian, as the sensor model of the Kalman filter (Lecture 6),

$$p(y \mid \mathbf{x}) = \mathcal{N}(y \mid \mathbf{w}^T\mathbf{x} + b, \sigma^2),$$

where $\mathbf{w}$ and b are parameters to determine.

Following the principle of maximum likelihood estimation (Lecture 5), we maximize the log-likelihood of the data $\mathbf{d} = \{(\mathbf{x}_i, y_i)\}$ with respect to $\mathbf{w}$ and b,

$$\begin{aligned}\log \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i, \mathbf{w}, b) &= \sum_{i=1}^{N} \log \mathcal{N}(y_i \mid \mathbf{w}^T \mathbf{x}_i + b, \sigma^2) \\ &= -\frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - (\mathbf{w}^T \mathbf{x}_i + b))^2 + \text{const.}\end{aligned}$$

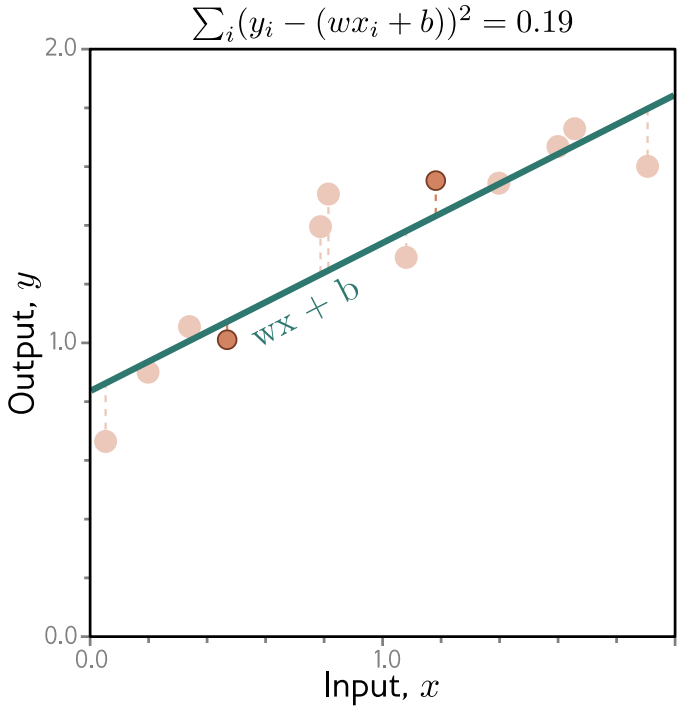
Following the principle of maximum likelihood estimation (Lecture 5), we maximize the log-likelihood of the data $\mathbf{d} = \{(\mathbf{x}_i, y_i)\}$ with respect to $\mathbf{w}$ and b,

$$\begin{aligned}\log \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i, \mathbf{w}, b) &= \sum_{i=1}^{N} \log \mathcal{N}(y_i \mid \mathbf{w}^T \mathbf{x}_i + b, \sigma^2) \\ &= -\frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - (\mathbf{w}^T \mathbf{x}_i + b))^2 + \text{const.}\end{aligned}$$

This amounts to minimizing the sum of squared errors,

$$\mathcal{L}(\mathbf{w}, b) = \sum_{i=1}^{N} (y_i - (\mathbf{w}^T \mathbf{x}_i + b))^2.$$

> Least squares is the maximum likelihood estimate of a linear model with Gaussian noise of fixed variance.



Minimizing the negative log-likelihood of a linear Gaussian model reduces to minimizing the sum of squared residuals.

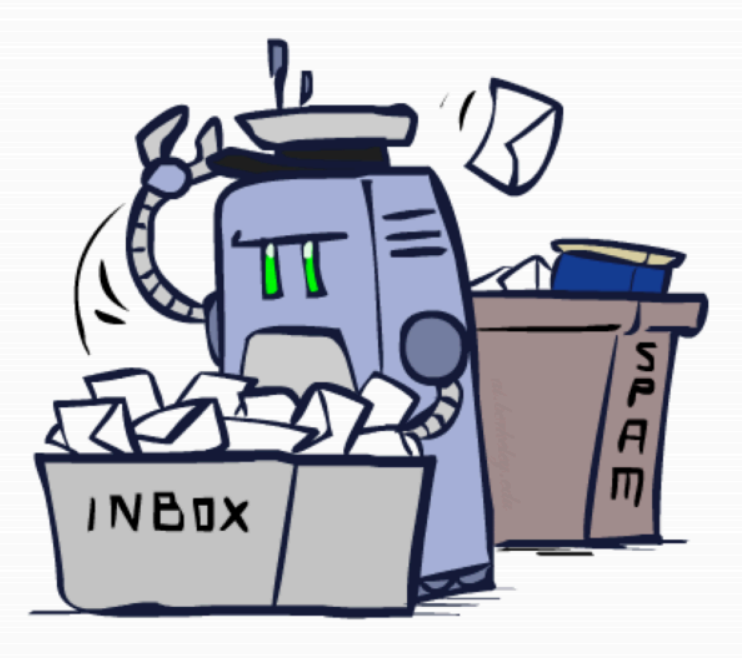
If we absorb the bias term b into the weight vector $\mathbf{w}$ by appending a constant feature equal to 1 to the input vector $\mathbf{x}$, the solution $\mathbf{w}^*$ is given analytically by

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y},$$

where $\mathbf{X}$ is the input matrix made of the stacked input vectors $\mathbf{x}_i$ (including the constant feature) and $\mathbf{y}$ is the output vector made of the output values y_i.

Logistic regression

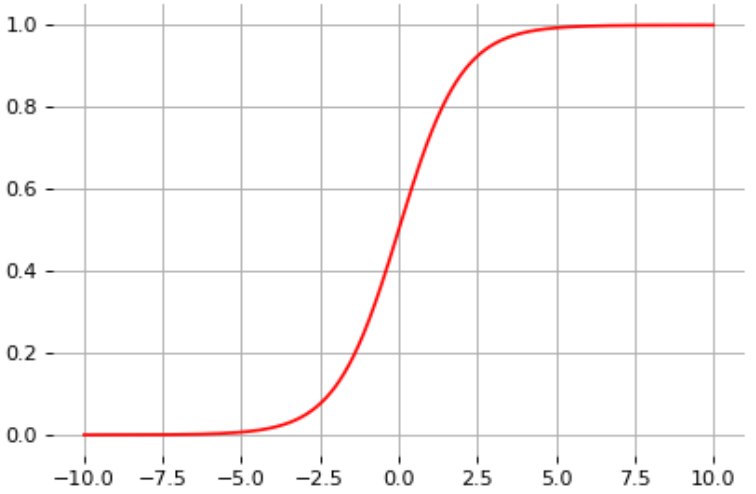
Let us now assume $y \in \{0, 1\}$.



Logistic regression models the conditional as

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b),$$

where the sigmoid activation function $\sigma(x) = \frac{1}{1+\exp(-x)}$ looks like a soft Heaviside step:



Following the principle of maximum likelihood estimation (Lecture 5), we have

$$
\begin{aligned}
&\arg\max_{\mathbf{w},b} P(\mathbf{d} \mid \mathbf{w}, b) \\
&= \arg\max_{\mathbf{w},b} \prod_{\mathbf{x}_i, y_i \in \mathbf{d}} P(Y = y_i \mid \mathbf{x}_i, \mathbf{w}, b) \\
&= \arg\max_{\mathbf{w},b} \prod_{\mathbf{x}_i, y_i \in \mathbf{d}} \sigma(\mathbf{w}^T \mathbf{x}_i + b)^{y_i} (1 - \sigma(\mathbf{w}^T \mathbf{x}_i + b))^{1 - y_i} \\
&= \arg\min_{\mathbf{w},b} \underbrace{\sum_{\mathbf{x}_i, y_i \in \mathbf{d}} -y_i \log \sigma(\mathbf{w}^T \mathbf{x}_i + b) - (1 - y_i) \log(1 - \sigma(\mathbf{w}^T \mathbf{x}_i + b))}_{\mathcal{L}(\mathbf{w},b) = \sum_i \ell(y_i, \hat{y}(\mathbf{x}_i; \mathbf{w}, b))}
\end{aligned}
$$

The loss ℓ is the **cross-entropy**, the negative log-probability of the observed class.

Unfortunately, there is no closed-form solution for the MLE of $\mathbf{w}$ and b.

Gradient descent

Let $\mathcal{L}(\theta)$ denote a loss function (e.g., the negative log-likelihood) defined over model parameters θ (e.g., $\mathbf{w}$ and b).

To minimize $\mathcal{L}(\theta)$, **gradient descent** relies on 1st-order Taylor approximations

$$\mathcal{L}(\theta + \epsilon) \approx \mathcal{L}(\theta) + \epsilon^T \nabla_\theta \mathcal{L}(\theta)$$

and adds a quadratic regularization term to ensure that the approximation is valid only in a neighborhood of θ, leading to the approximation

$$\hat{\mathcal{L}}(\epsilon; \theta) = \mathcal{L}(\theta) + \epsilon^T \nabla_\theta \mathcal{L}(\theta) + \frac{1}{2\alpha} |\epsilon|^2,$$

where $\alpha > 0$ controls the size of the neighborhood.

The approximation $\hat{\mathcal{L}}(\epsilon;\theta)$ is minimized where its gradient vanishes,

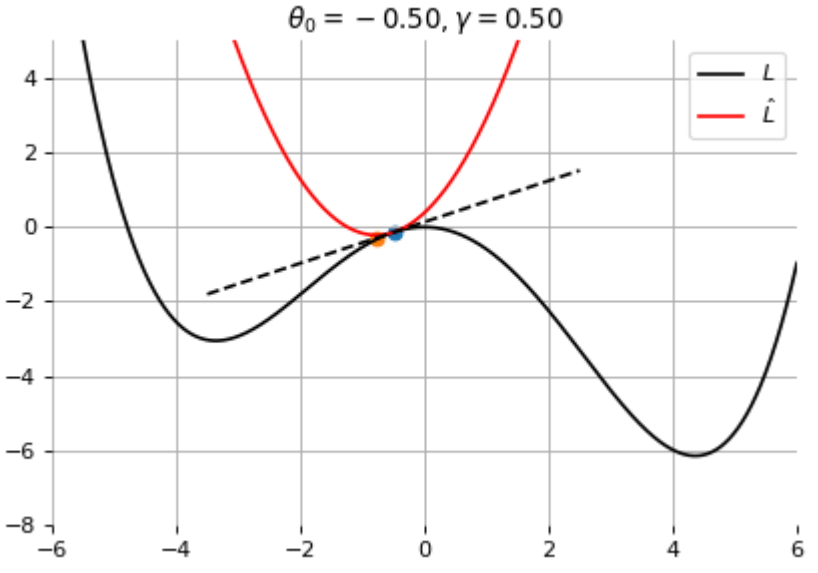
$$\nabla_\epsilon \hat{\mathcal{L}}(\epsilon;\theta) = \nabla_\theta \mathcal{L}(\theta) + \frac{1}{\alpha}\epsilon = 0,$$

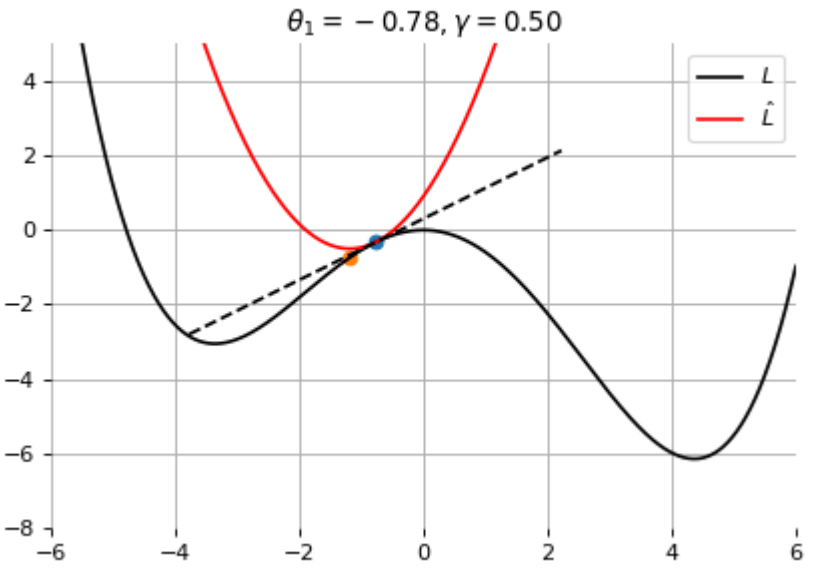
that is, for the step $\epsilon = -\alpha \nabla_\theta \mathcal{L}(\theta)$.

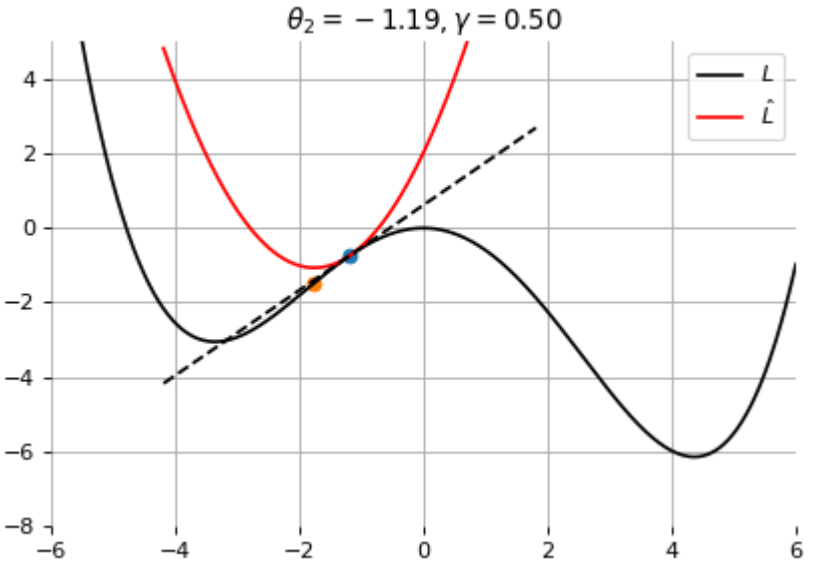
Starting from initial parameters θ_0, gradient descent repeats this step,

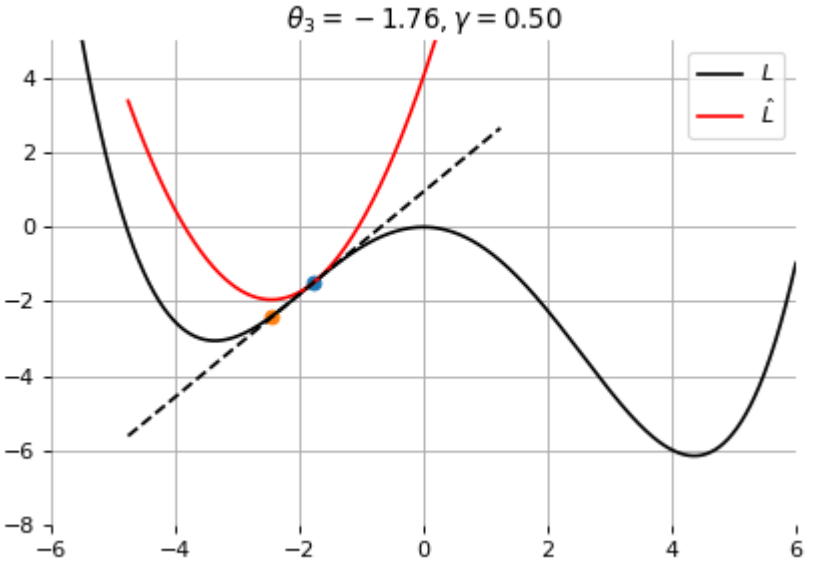
$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t),$$

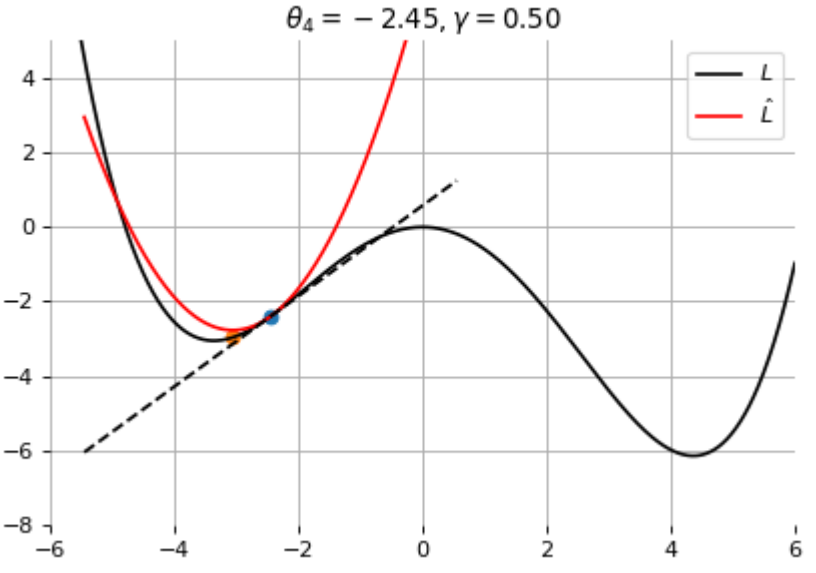
where α is the **learning rate**.

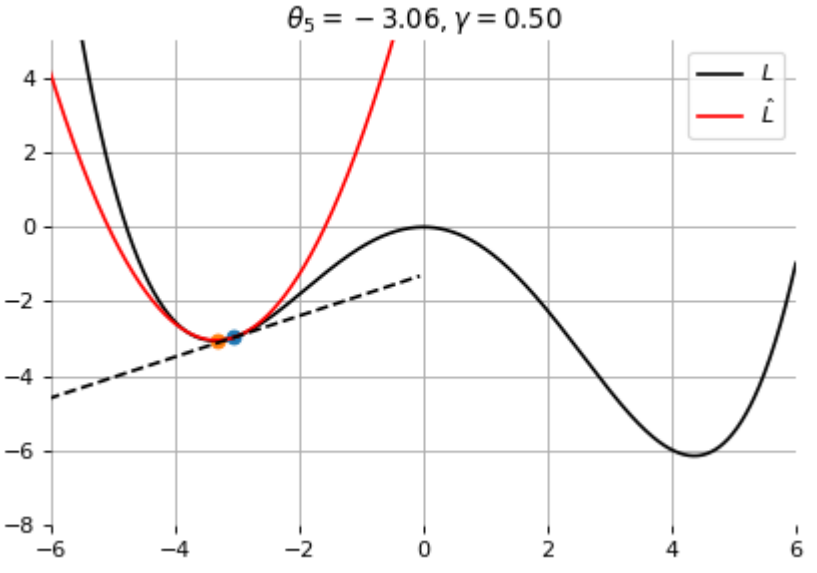

$\theta_0 = -0.50, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6

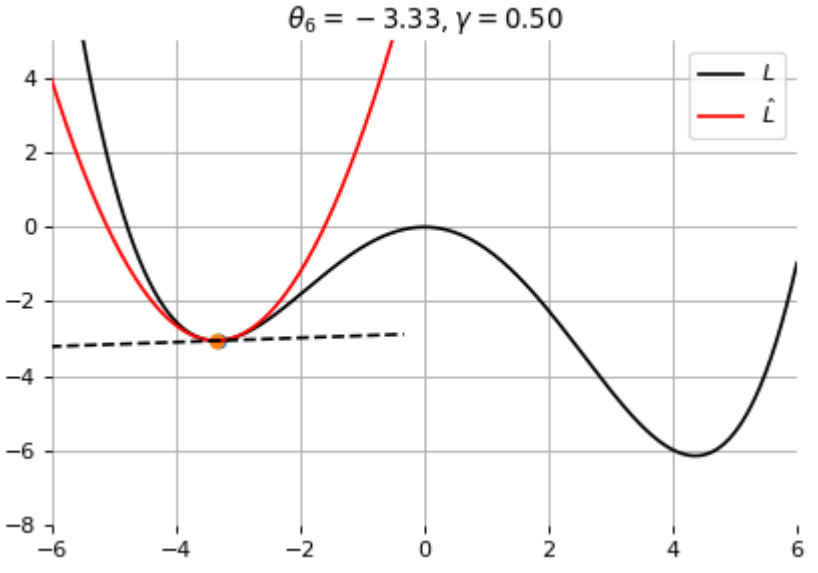

$\theta_1 = -0.78, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6

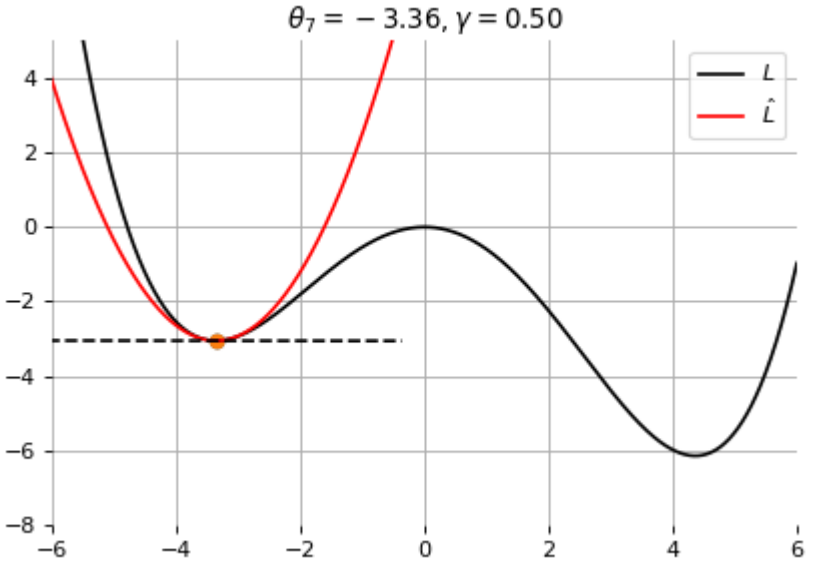

$\theta_2 = -1.19, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6


$\theta_3 = -1.76, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6


$\theta_4 = -2.45, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6


$\theta_5 = -3.06, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6


$\theta_6 = -3.33, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6


$\theta_7 = -3.36, \gamma = 0.50$
L
$\hat{L}$
4
2
0
−2
−4
−6
−8
−6
−4
−2
0
2
4
6

Up to a constant factor, the loss is an average over the N pairs,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, \hat{y}(\mathbf{x}_i; \theta)),$$

and each step of gradient descent costs a pass over the dataset.

Stochastic gradient descent averages the gradient over a random **minibatch** $\mathcal{B}$ of $B \ll N$ pairs instead,

$$\theta_{t+1} = \theta_t - \alpha \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta \ell(y_i, \hat{y}(\mathbf{x}_i; \theta_t)).$$

The estimate is noisy but right on average, at a fraction B/N of the cost.

Training loop

function TRAIN(d, f, α, B, $epochs$) **returns** the parameters θ

  $\theta \leftarrow$ random values

  repeat $epochs$ times

    for each minibatch of B pairs of d, in random order **do**

      $L \leftarrow$ the average of $\ell(y, f(x; \theta))$ over the pairs (x, y) of the minibatch

      $g \leftarrow \nabla_\theta L$, the gradient of the loss

      $\theta \leftarrow \theta - \alpha\, g$

  return θ

Each line comes from the previous slides.

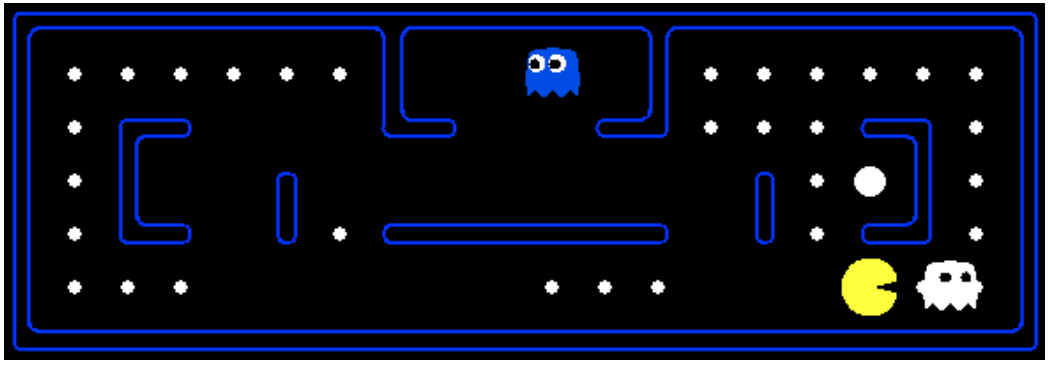
(Step-by-step code example)

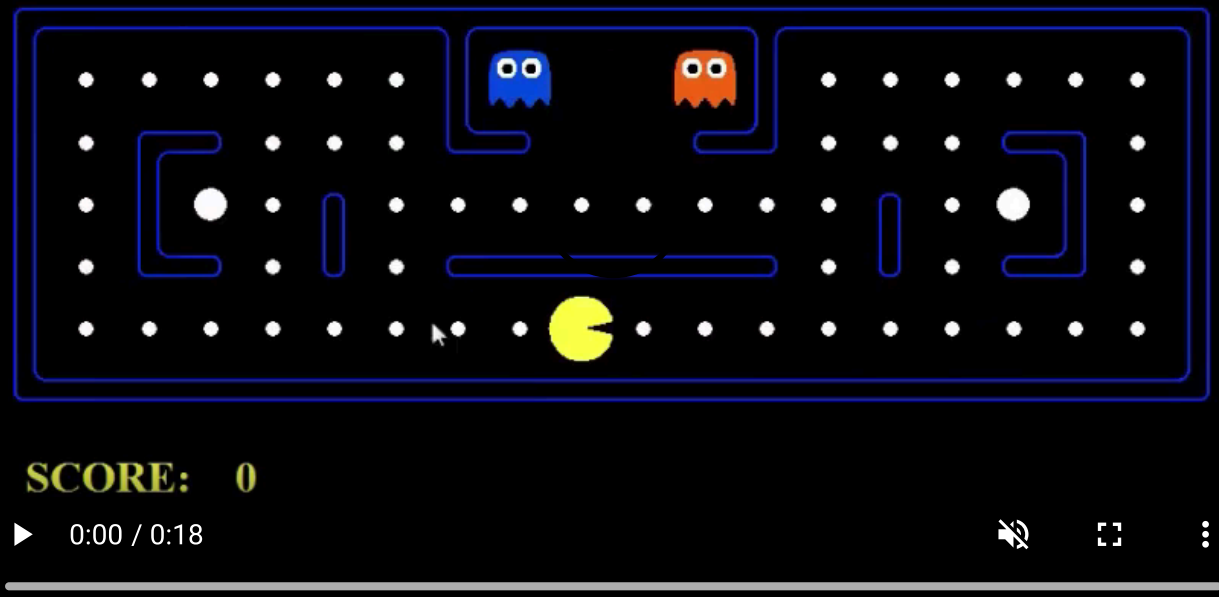
`demo/lec7/spiral.ipynb`

Example: imitation learning in Pacman

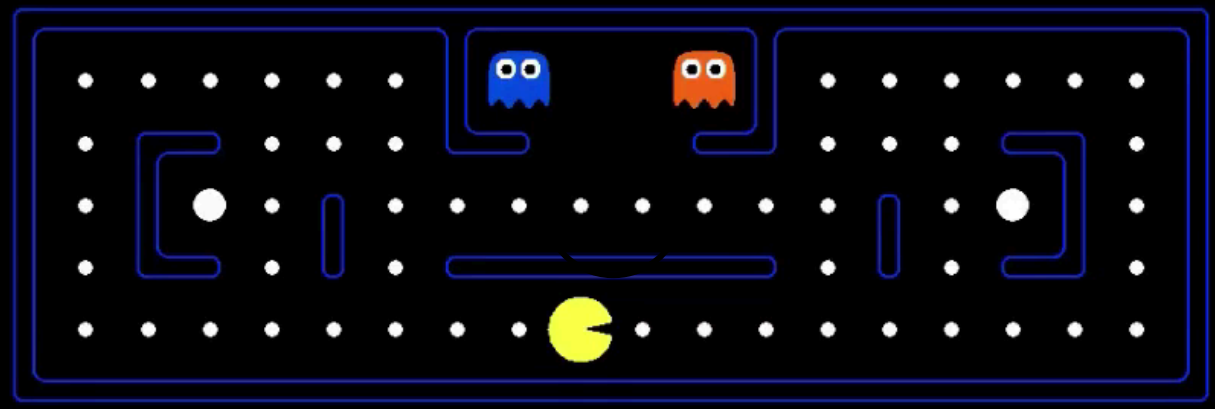
Can we learn to play Pacman only by watching an expert, e.g. the minimax agent of Lecture 3?

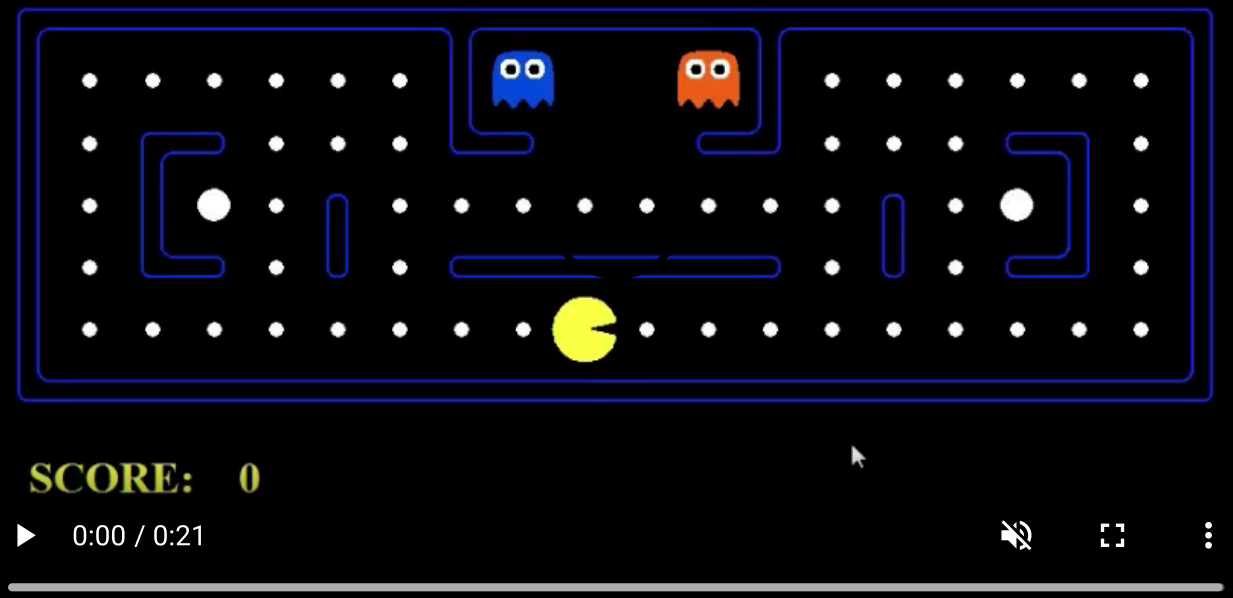
- At each step, the expert perceives e_t and acts a_t. Pacman's world is fully observable, so $e_t = s_t$.
- Each step gives a pair $(\mathbf{x}, y)$, with features $\mathbf{x} = g(e_t)$ and the expert's action $y = a_t$.
- We learn a simple reflex agent (Lecture 1), $a_t = \pi_\theta(e_t) = f(g(e_t); \theta)$ with $f : \mathbb{R}^d \to \mathcal{A}$, whose condition-action rules are learned rather than written.
- This is a multiclass classification problem.





The agent observes a very good Minimax-based agent for two games and updates its weight vectors as data are collected.

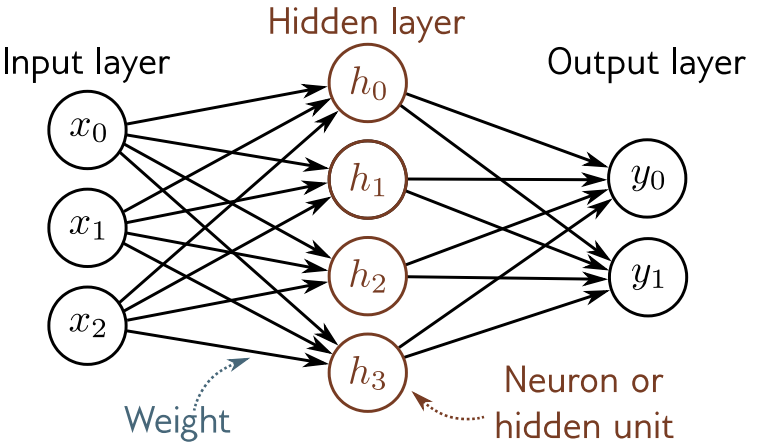

SCORE: 0
0:00 / 0:18



After two training episodes, the ML-based agents plays.
No more Minimax!

Deep learning

Shallow networks



A shallow network is a function

$$f : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$$

that maps multi-dimensional inputs $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to multi-dimensional outputs $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$ through a hidden layer $\mathbf{h} = [h_0, h_1, ..., h_{q-1}] \in \mathbb{R}^q$.

The hidden layer is a non-linear transformation of the input, such that

$$h_j = \sigma\left(\sum_{i=0}^{d_{\text{in}}-1} w_{ji}x_i + b_j\right)$$

for $j = 0, ..., q-1$, where w_{ji} and b_j ($i = 0, ..., d_{\text{in}}-1$, $j = 0, ..., q-1$) are the model parameters and σ is an **activation function**, such as the sigmoid of logistic regression.

The output layer is a linear transformation of the hidden layer, such that

$$y_k = \sum_{j=0}^{q-1} v_{kj}h_j + c_k$$

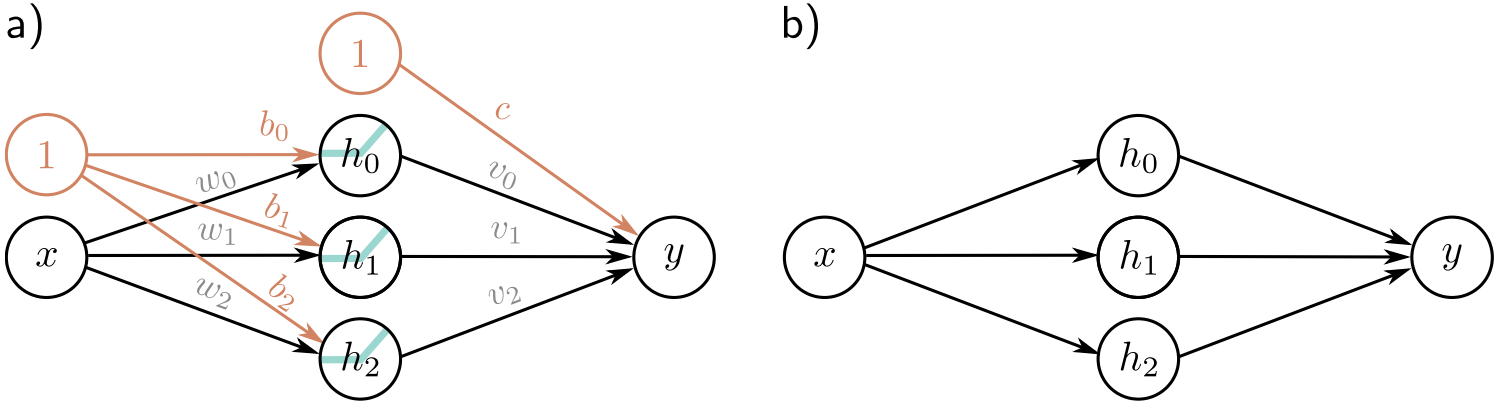
for $k = 0, ..., d_{\text{out}}-1$, where v_{kj} and c_k ($j = 0, ..., q-1$, $k = 0, ..., d_{\text{out}}-1$) are the model parameters.

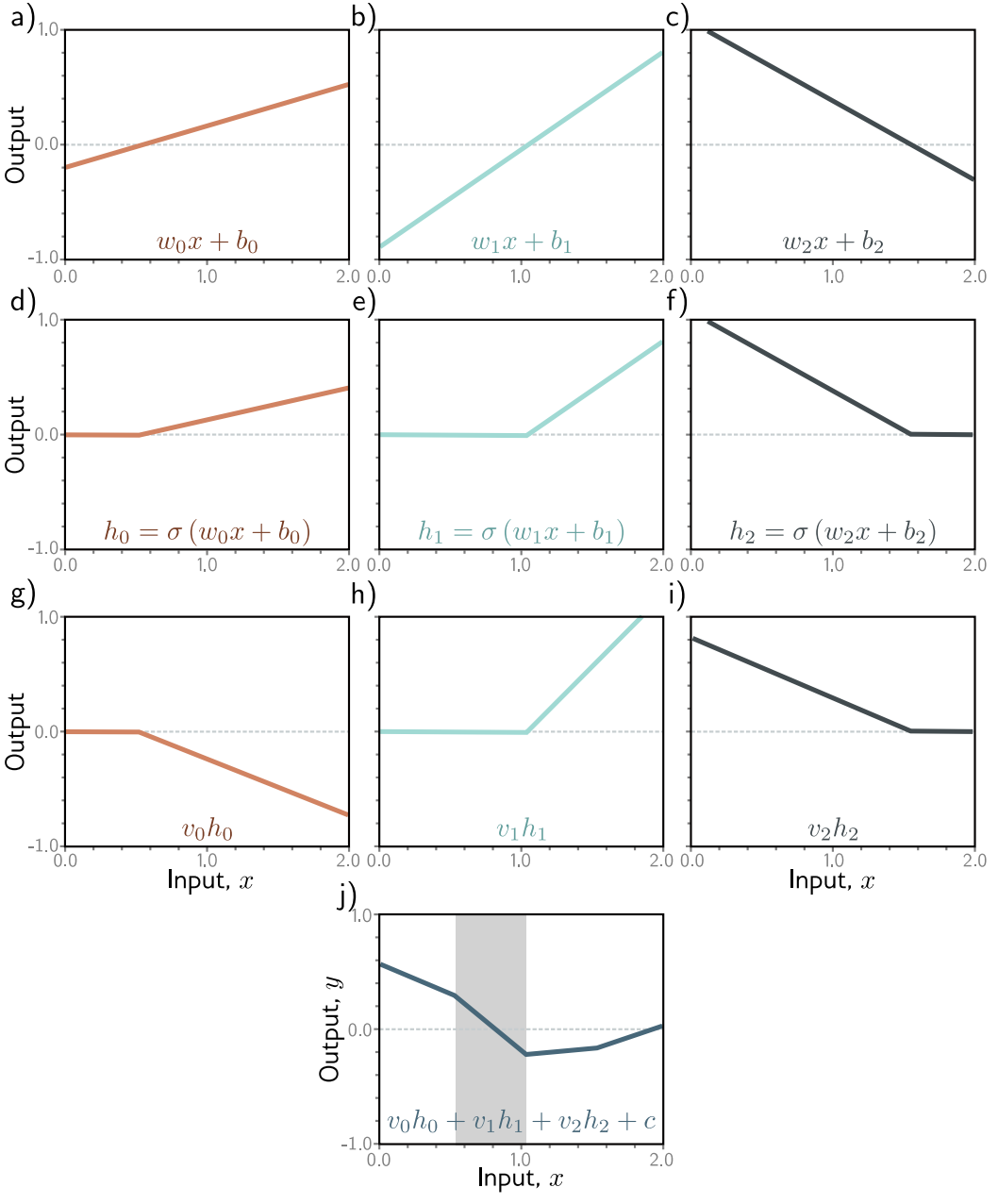
Single-input single-output networks

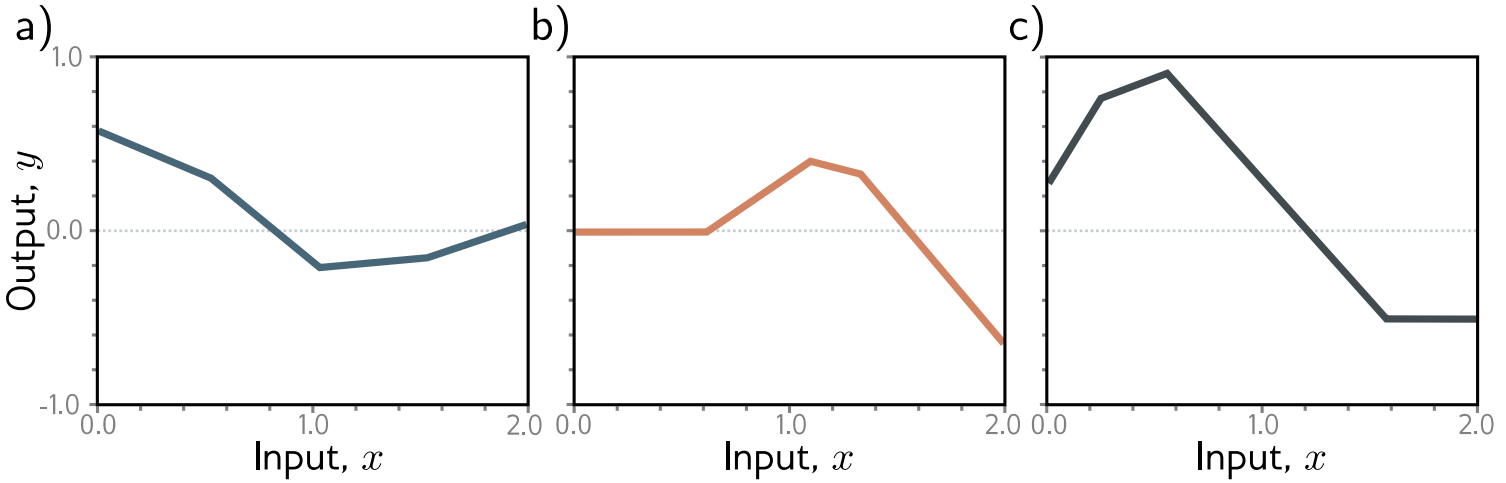
We first consider the case where $d_{\text{in}} = 1$ and $d_{\text{out}} = 1$ for the single-input single-output network

$$y = v_0\sigma(w_0x + b_0) + v_1\sigma(w_1x + b_1) + v_2\sigma(w_2x + b_2) + c$$

where w_0, w_1, w_2, b_0, b_1, b_2, v_0, v_1, v_2 and c are the model parameters and where the activation function σ is $\text{ReLU}(\cdot) = \max(0, \cdot)$.




a)
b)
c)
d)
e)
f)
g)
h)
i)
j)
Output
Output, y
Input, x
$w_0x + b_0$
$w_1x + b_1$
$w_2x + b_2$
$h_0 = \sigma(w_0x + b_0)$
$h_1 = \sigma(w_1x + b_1)$
$h_2 = \sigma(w_2x + b_2)$
v_0h_0
v_1h_1
v_2h_2
$v_0h_0 + v_1h_1 + v_2h_2 + c$
1.0
0.0
-1.0
0.0
1.0
2.0

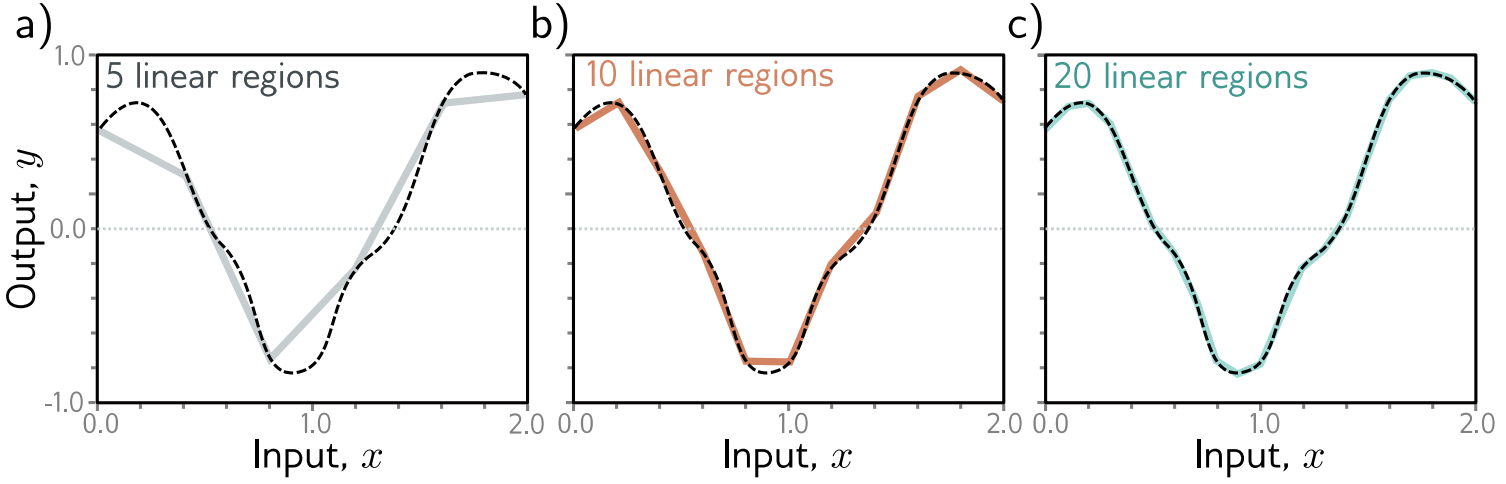


This shallow network defines a family of piecewise linear functions where the positions of the joints, the slopes and the heights of the functions are determined by the 10 parameters w_0, w_1, w_2, b_0, b_1, b_2, v_0, v_1, v_2 and c.

Universal approximation theorem

The **universal approximation theorem** states that any continuous function on a closed and bounded domain can be approximated arbitrarily well by a shallow network with sufficiently many hidden units, provided that the activation function is not a polynomial.

For example, for 1d input and 1d output, any continuous function $f : [a, b] \to \mathbb{R}$ can be approximated to arbitrary accuracy by a piecewise linear function with enough linear segments, which can be represented by a shallow network with enough hidden units.

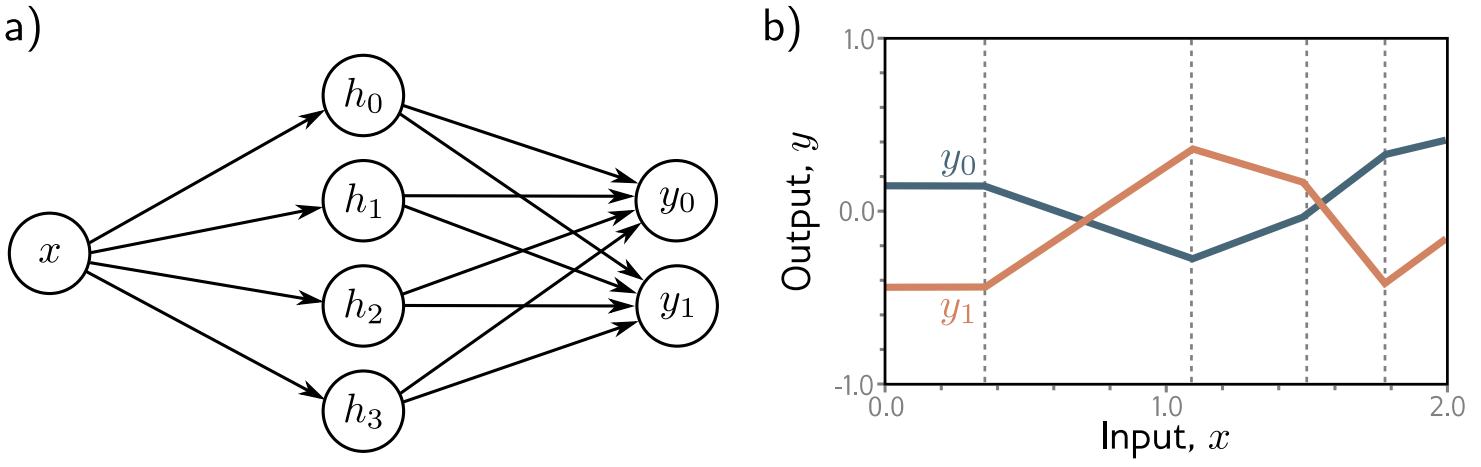

a)
5 linear regions
b)
10 linear regions
c)
20 linear regions
Output, y
Input, x
Input, x
Input, x
1.0
0.0
-1.0
0.0
1.0
2.0

Multivariate outputs

To extend the network to multivariate outputs $\mathbf{y} = [y_0, y_1, .., y_{d_{\text{out}}-1}]$, we simply add more output units as linear combinations of the hidden units.

For example, a network with two output units y_0 and y_1 might have the following structure:

$$
\begin{aligned}
h_0 &= \sigma\left(w_0 x + b_0\right) \\
h_1 &= \sigma\left(w_1 x + b_1\right) \\
h_2 &= \sigma\left(w_2 x + b_2\right) \\
h_3 &= \sigma\left(w_3 x + b_3\right) \\
y_0 &= v_{00} h_0 + v_{01} h_1 + v_{02} h_2 + v_{03} h_3 + c_0 \\
y_1 &= v_{10} h_0 + v_{11} h_1 + v_{12} h_2 + v_{13} h_3 + c_1
\end{aligned}
$$

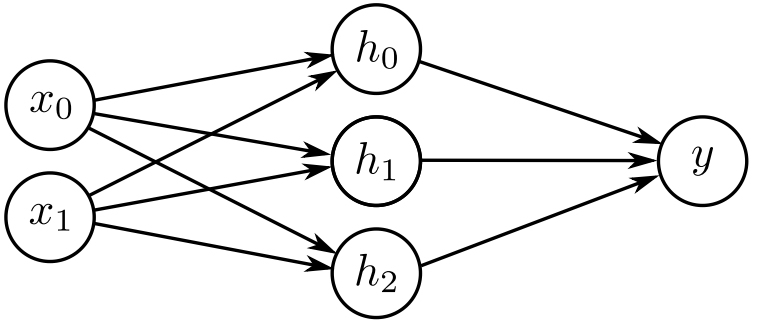


a) With two output units, the network can model two functions of the input $\boldsymbol{x}$.

b) The four joints of these functions are constrained to be at the same positions, but the slopes and heights of the functions can vary independently.

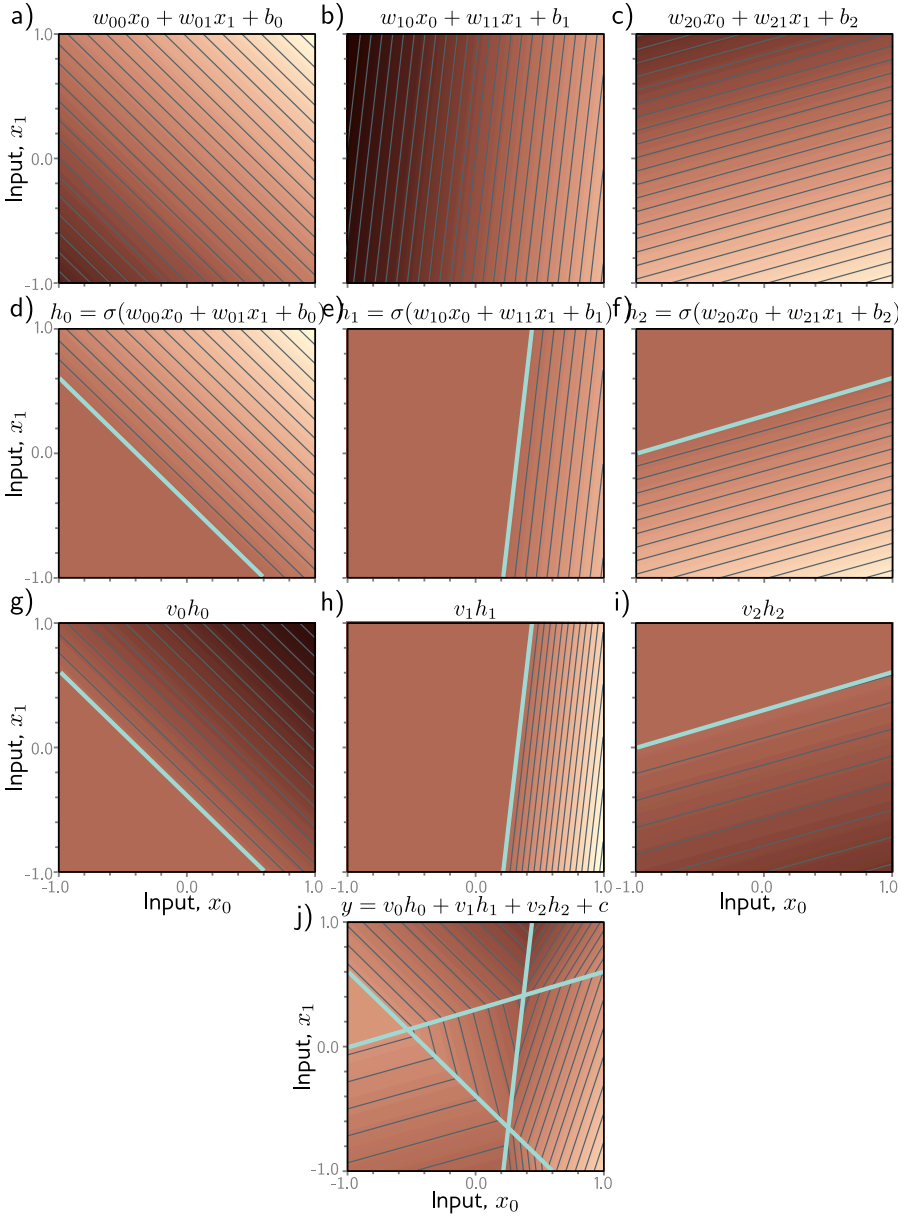
Multivariate inputs

To extend the network to multivariate inputs $\mathbf{x} = [x_0, x_1, ..., x_{d_{\text{in}}-1}]$, we extend the linear relations between the input and the hidden units.



For example, a network with two inputs $\mathbf{x} = [x_0, x_1]$ might have three hidden units h_0, h_1 and h_2 defined as

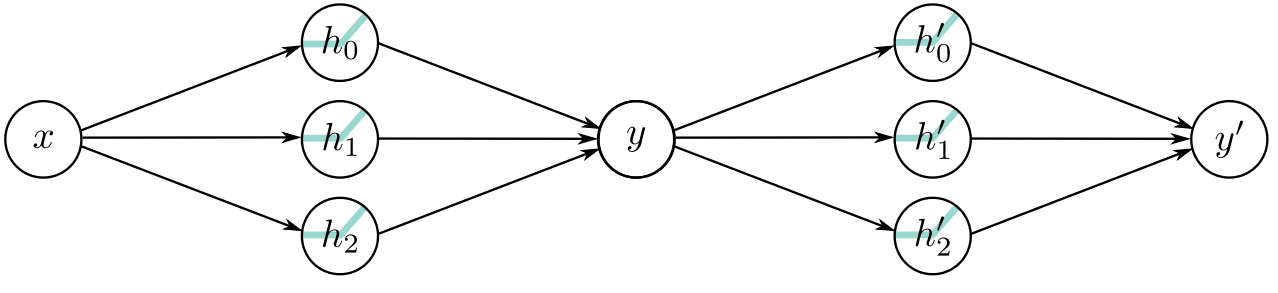
$$
\begin{aligned}
h_0 &= \sigma\left(w_{00}x_0 + w_{01}x_1 + b_0\right) \\
h_1 &= \sigma\left(w_{10}x_0 + w_{11}x_1 + b_1\right) \\
h_2 &= \sigma\left(w_{20}x_0 + w_{21}x_1 + b_2\right).
\end{aligned}
$$


a) $w_{00}x_0 + w_{01}x_1 + b_0$
b) $w_{10}x_0 + w_{11}x_1 + b_1$
c) $w_{20}x_0 + w_{21}x_1 + b_2$
d) $h_0 = \sigma(w_{00}x_0 + w_{01}x_1 + b_0)$
e) $h_1 = \sigma(w_{10}x_0 + w_{11}x_1 + b_1)$
f) $h_2 = \sigma(w_{20}x_0 + w_{21}x_1 + b_2)$
g) v_0h_0
h) v_1h_1
i) v_2h_2
j) $y = v_0h_0 + v_1h_1 + v_2h_2 + c$
Input, x_0
Input, x_1
1.0
0.0
-1.0

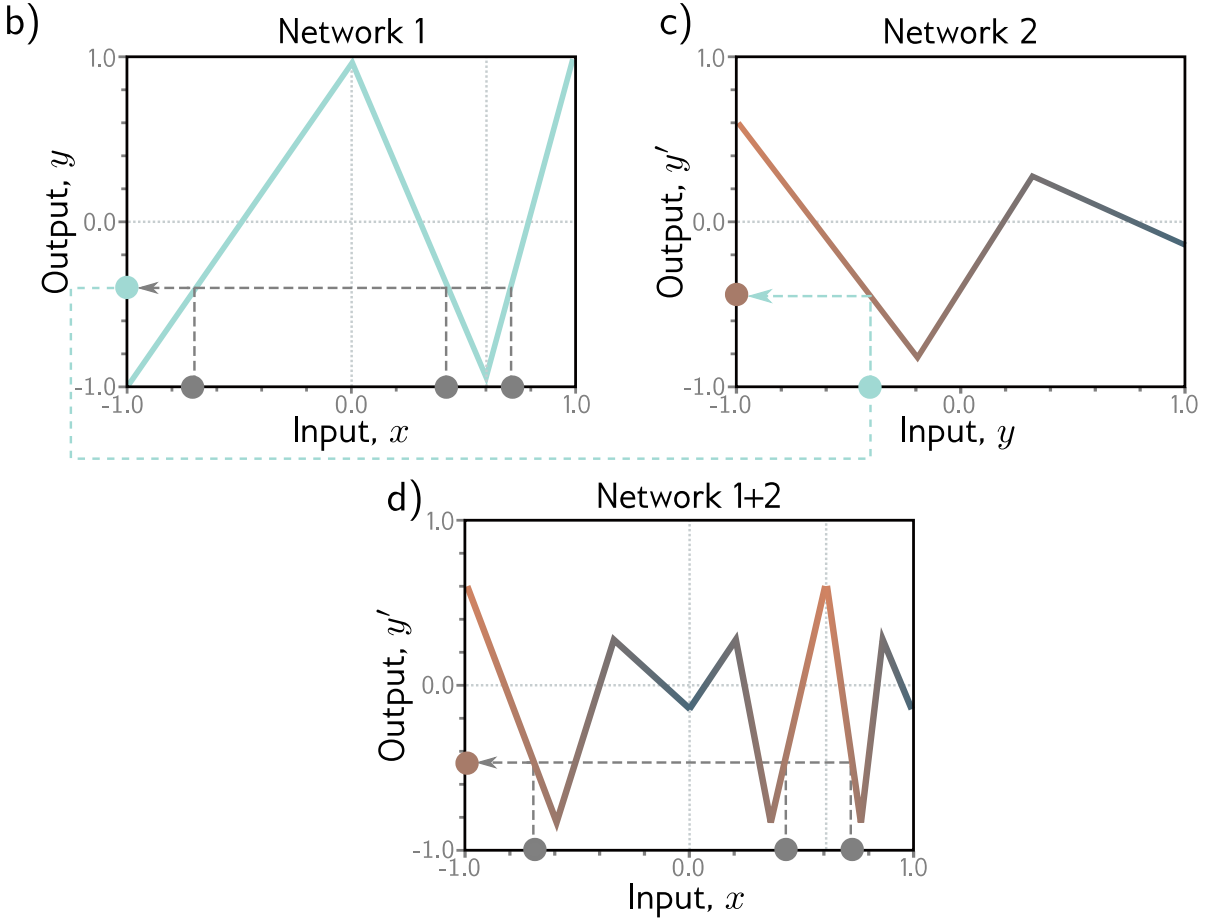
Deep networks

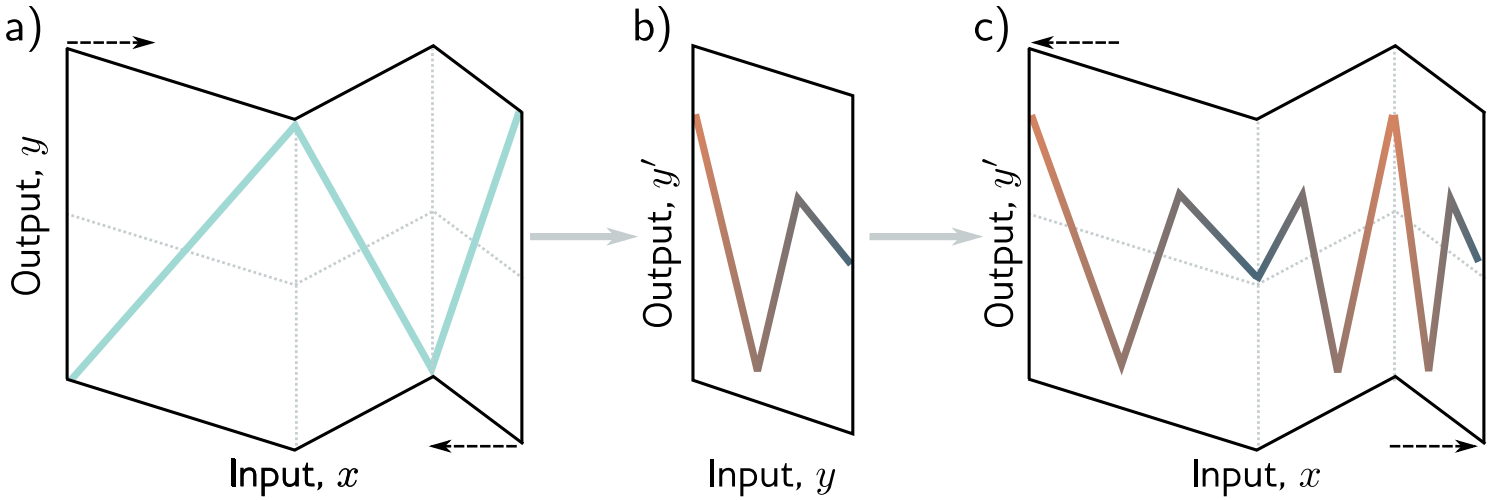
We first consider the composition of two shallow networks, where the output of the first network is fed as input to the second network as

$$
\begin{aligned}
h_0 &= \sigma\left(w_0 x + b_0\right) \\
h_1 &= \sigma\left(w_1 x + b_1\right) \\
h_2 &= \sigma\left(w_2 x + b_2\right) \\
y &= v_0 h_0 + v_1 h_1 + v_2 h_2 + c \\
h'_0 &= \sigma\left(w'_0 y + b'_0\right) \\
h'_1 &= \sigma\left(w'_1 y + b'_1\right) \\
h'_2 &= \sigma\left(w'_2 y + b'_2\right) \\
y' &= v'_0 h'_0 + v'_1 h'_1 + v'_2 h'_2 + c'.
\end{aligned}
$$



With $\mathbf{ReLU}$ activation functions, this network also describes a family of piecewise linear functions. However, each linear region defined by the hidden units of the first network is further divided by the hidden units of the second network.

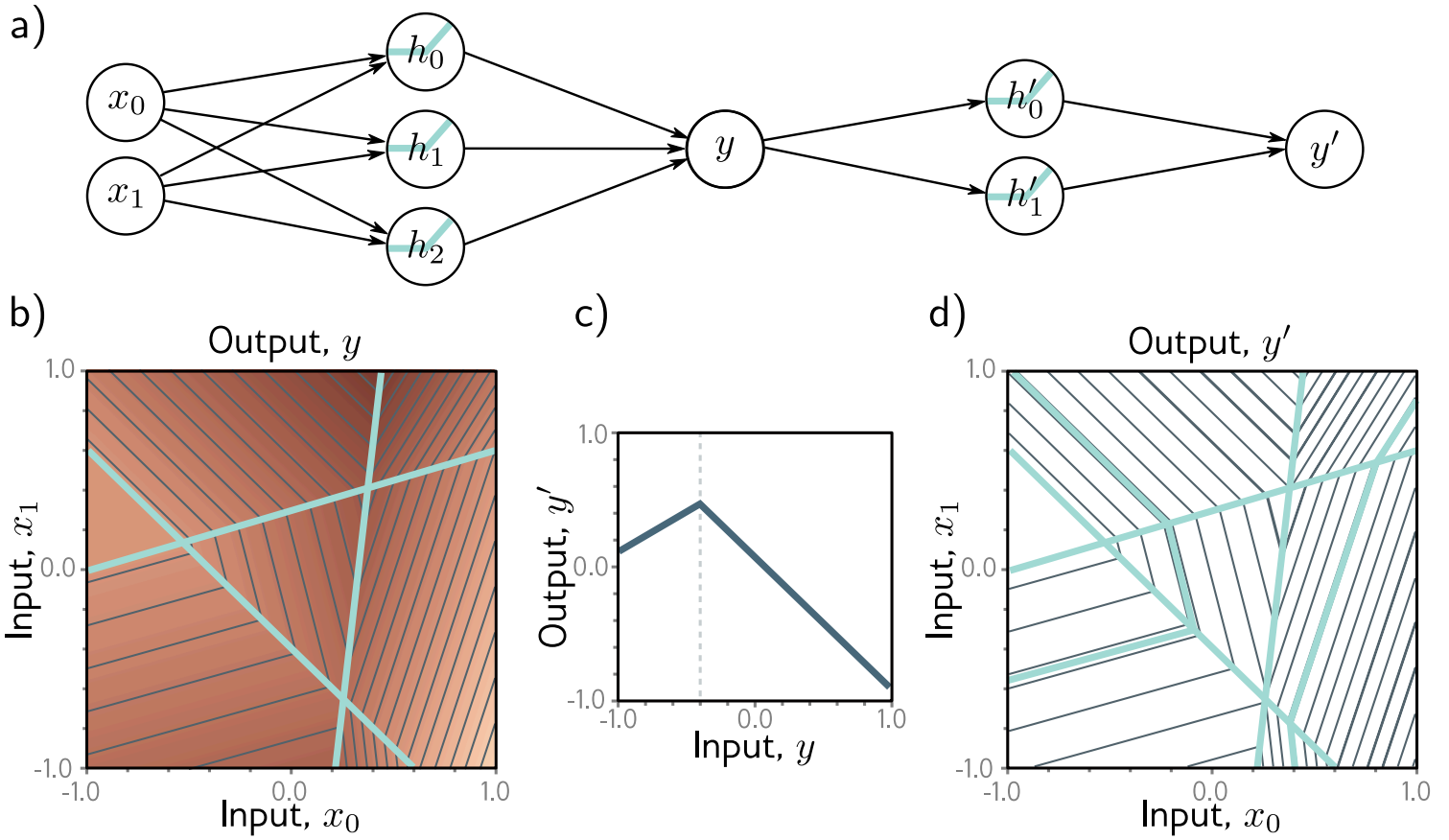




The composition of two shallow networks can be interpreted as a folding operation in three steps:

a) The first network folds the input space back on itself.
b) The second network applies its function to the folded space.
c) The final output is revealed by unfolding the folded space.

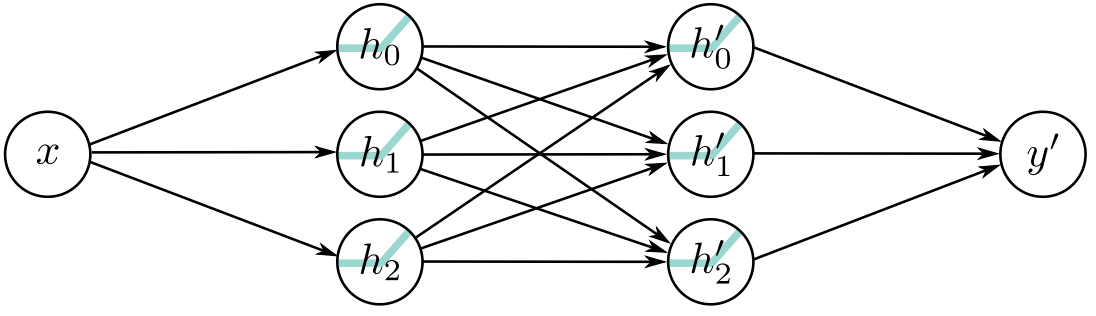
Similarly, composing a multivariate shallow network with a shallow network further divides the input space into more linear regions.



From composing shallow networks to deep networks

Since the operation from $[h_0, h_1, h_2]$ to y is linear and the operation from y to $[h'_0, h'_1, h'_2]$ is also linear, their composition in series is linear.

It follows that the composition of the two shallow networks is a special case of a deep network with two hidden layers.



Mathematically, the first layer is defined as

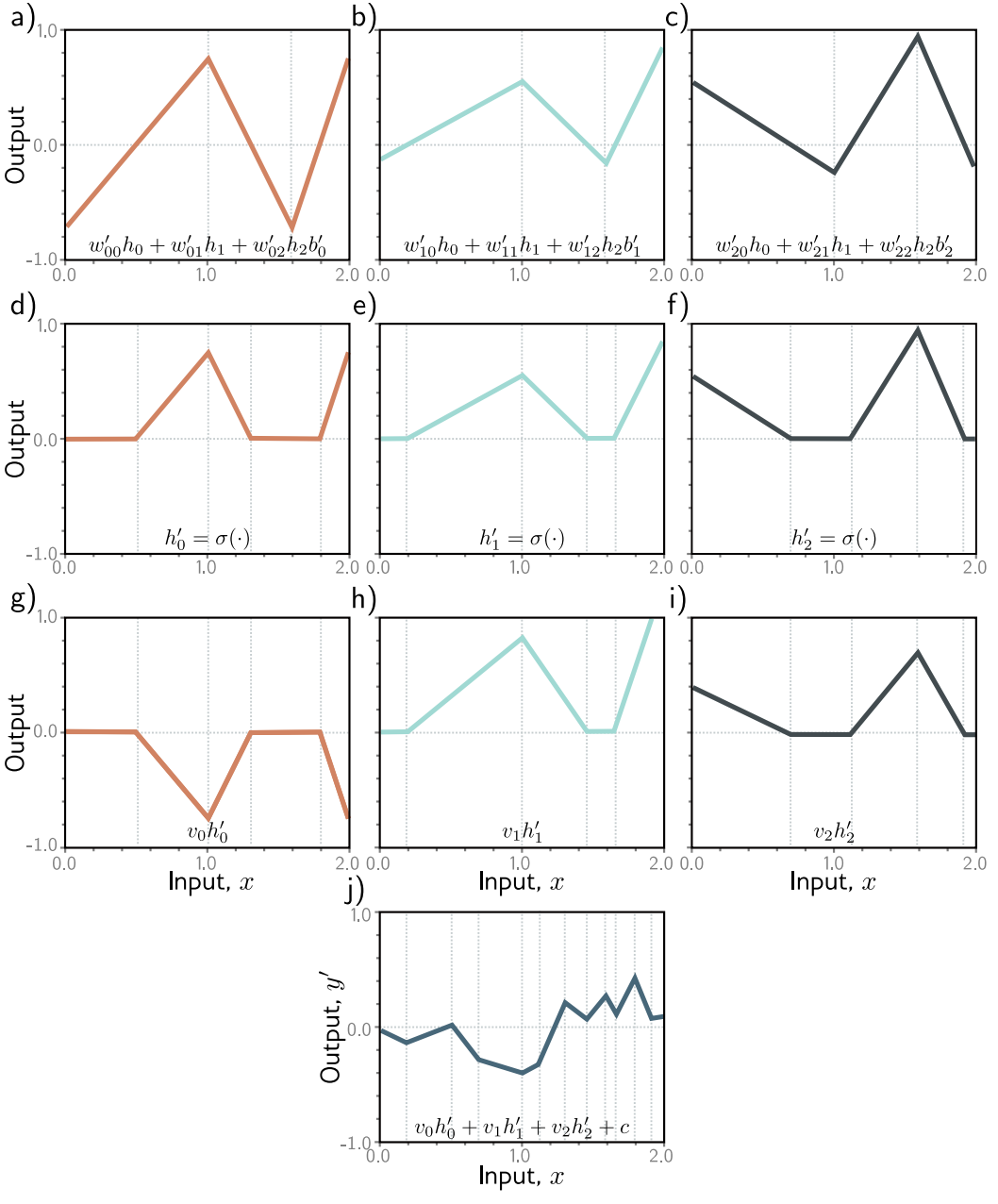
$$
\begin{aligned}
h_0 &= \sigma\left(w_0 x + b_0\right) \\
h_1 &= \sigma\left(w_1 x + b_1\right) \\
h_2 &= \sigma\left(w_2 x + b_2\right),
\end{aligned}
$$

while the second layer is defined from the outputs of the first layer as

$$
\begin{aligned}
h'_0 &= \sigma\left(w'_{00} h_0 + w'_{01} h_1 + w'_{02} h_2 + b'_0\right) \\
h'_1 &= \sigma\left(w'_{10} h_0 + w'_{11} h_1 + w'_{12} h_2 + b'_1\right) \\
h'_2 &= \sigma\left(w'_{20} h_0 + w'_{21} h_1 + w'_{22} h_2 + b'_2\right),
\end{aligned}
$$

and the output as

$$
y = v_0 h'_0 + v_1 h'_1 + v_2 h'_2 + c.
$$


a)
b)
c)
d)
e)
f)
g)
h)
i)
j)
Output
Output, y'
Input, x
$w'_{00}h_0 + w'_{01}h_1 + w'_{02}h_2b'_0$
$w'_{10}h_0 + w'_{11}h_1 + w'_{12}h_2b'_1$
$w'_{20}h_0 + w'_{21}h_1 + w'_{22}h_2b'_2$
$h'_0 = \sigma(\cdot)$
$h'_1 = \sigma(\cdot)$
$h'_2 = \sigma(\cdot)$
$v_0h'_0$
$v_1h'_1$
$v_2h'_2$
$v_0h'_0 + v_1h'_1 + v_2h'_2 + c$
1.0
0.0
-1.0
0.0
1.0
2.0

Multilayer perceptron

We first note that the computation of a hidden layer with q hidden units can be written in matrix form as

$$\mathbf{h} = \begin{bmatrix} h_0 \\ h_1 \\ \vdots \\ h_{q-1} \end{bmatrix} = \sigma\left(\begin{bmatrix} w_{00} & w_{01} & \cdots & w_{0(d_{\text{in}}-1)} \\ w_{10} & w_{11} & \cdots & w_{1(d_{\text{in}}-1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{(q-1)0} & w_{(q-1)1} & \cdots & w_{(q-1)(d_{\text{in}}-1)} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{d_{\text{in}}-1} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{q-1} \end{bmatrix} \right)$$
$$= \sigma(\mathbf{W}^T\mathbf{x} + \mathbf{b})$$

where $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is the input vector, $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times q}$ is the weight matrix of the hidden layer and $\mathbf{b} \in \mathbb{R}^q$ is the bias vector.

Using this notation, layers can be composed in series to form a deep network with K hidden layers such that

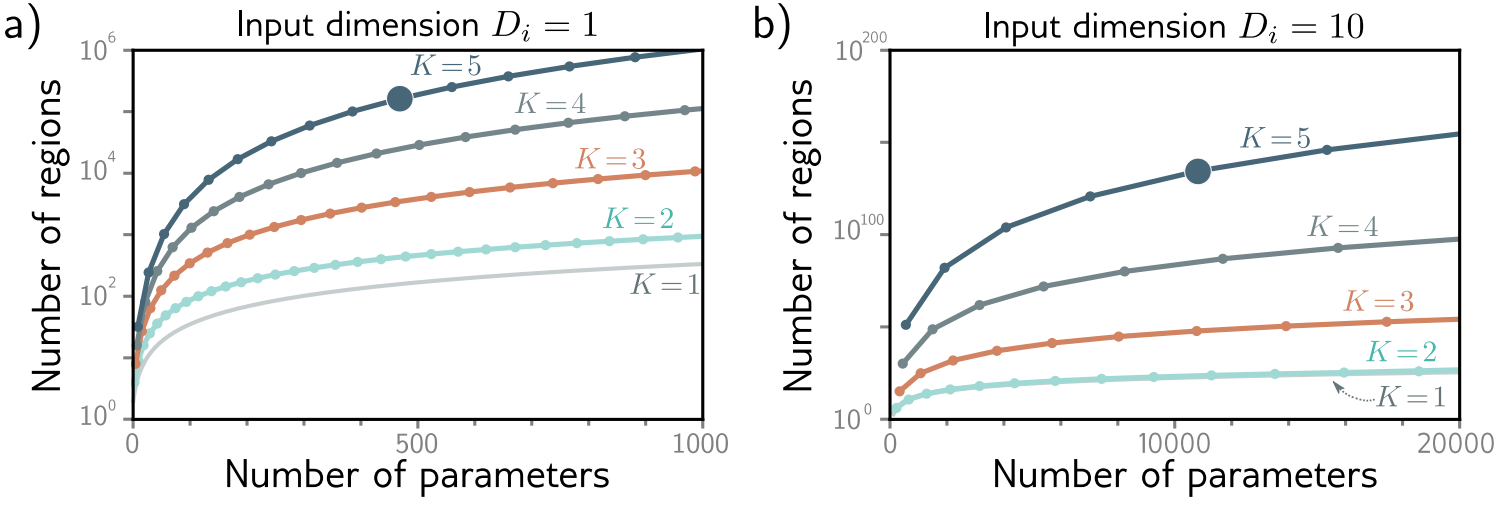
$$
\begin{aligned}
\mathbf{h}_0 &= \mathbf{x} \\
\mathbf{h}_1 &= \sigma(\mathbf{W}_1^T \mathbf{h}_0 + \mathbf{b}_1) \\
\mathbf{h}_2 &= \sigma(\mathbf{W}_2^T \mathbf{h}_1 + \mathbf{b}_2) \\
&\vdots \\
\mathbf{h}_K &= \sigma(\mathbf{W}_K^T \mathbf{h}_{K-1} + \mathbf{b}_K) \\
\mathbf{y} &= \sigma_{\text{out}}(\mathbf{W}_y^T \mathbf{h}_K + \mathbf{b}_y)
\end{aligned}
$$

where $\mathbf{W}_k \in \mathbb{R}^{q_{k-1} \times q_k}$ is the weight matrix of the k-th hidden layer and $\mathbf{b}_k \in \mathbb{R}^{q_k}$ is the bias vector of the k-th layer. The output activation σ_{out} depends on the task.

The resulting function $f(\mathbf{x}; \theta)$ is a highly non-linear mapping from inputs $\mathbf{x}$ to outputs $\mathbf{y}$, where θ denotes the set of all model parameters. This architecture is known as the **multilayer perceptron**.

A shallow network with one input, one output, and q hidden units can create up to $q+1$ linear regions and is defined by $O(q)$ parameters.

A deep network with K hidden layers, each with q hidden units, can create up to $(q+1)^K$ linear regions and is defined by $O(Kq^2)$ parameters.



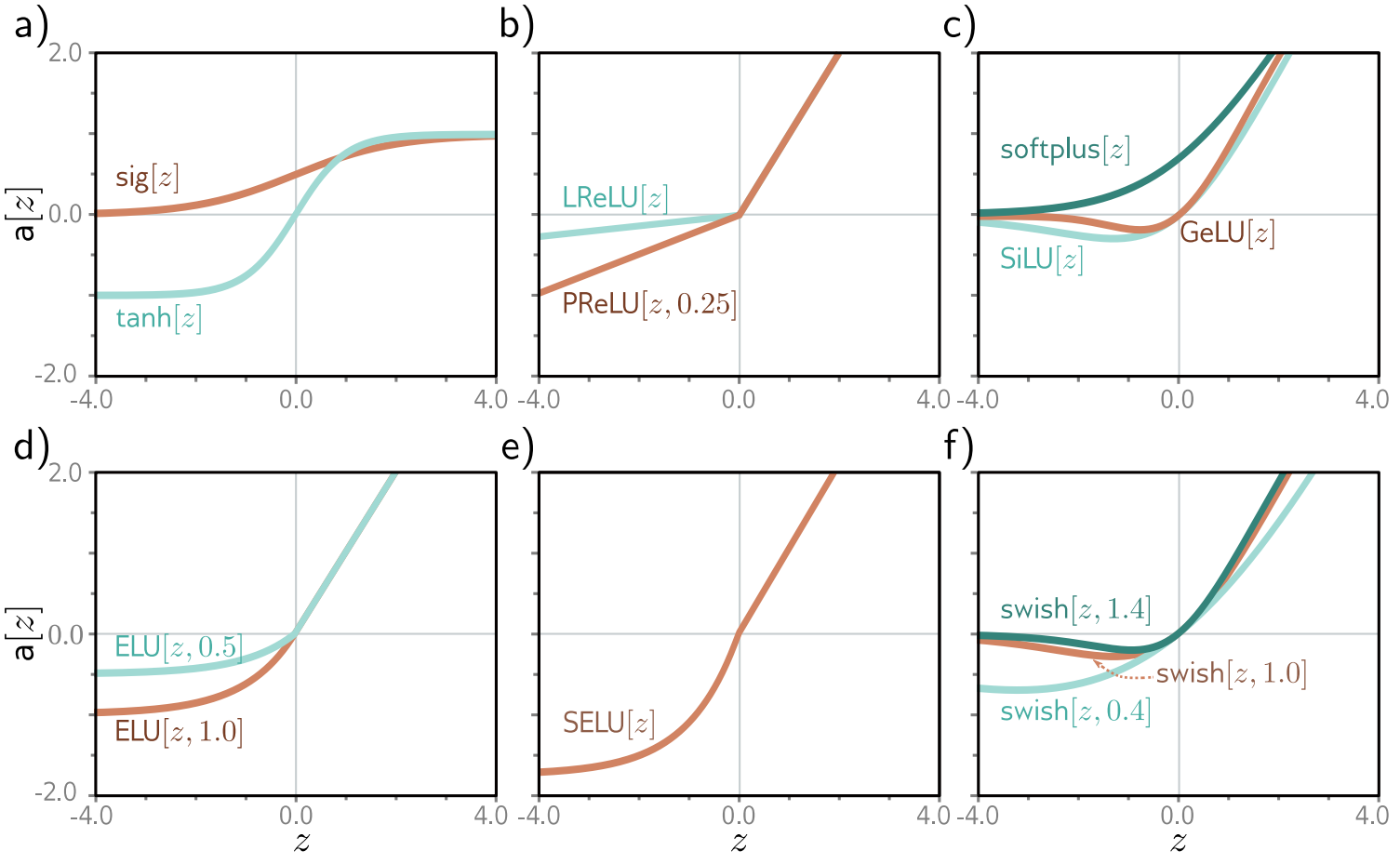
Activation functions

Activation functions $\sigma : \mathbb{R} \to \mathbb{R}$ introduce non-linearities in the network, enabling the model to learn complex functions.

Common activation functions include:

- Sigmoid: $\sigma(x) = \frac{1}{1+\exp(-x)}$, which maps inputs to the range $(0, 1)$.
- Hyperbolic tangent (tanh): $\sigma(x) = \tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$, which maps inputs to the range $(-1, 1)$.
- Rectified Linear Unit (ReLU): $\sigma(x) = \max(0, x)$, which introduces sparsity, and whose derivative, unlike that of the sigmoid, does not vanish for large inputs.

The choice of the activation function σ is crucial for the expressiveness of the network and the optimization of the model parameters.


a)
b)
c)
d)
e)
f)
a[z]
z
2.0
0.0
-2.0
-4.0
0.0
4.0
sig[z]
tanh[z]
LReLU[z]
PReLU[z, 0.25]
softplus[z]
GeLU[z]
SiLU[z]
ELU[z, 0.5]
ELU[z, 1.0]
SELU[z]
swish[z, 1.4]
swish[z, 1.0]
swish[z, 0.4]

Output layers

The output layer $\mathbf{y} = \sigma_{\text{out}}(\mathbf{W}_y^T \mathbf{h}_K + \mathbf{b}_y)$ depends on the task.

- For regression, it has d_{out} units and σ_{out} is the identity.
- For binary classification, it has one unit and σ_{out} is the sigmoid, whose output is $P(Y = 1 \mid \mathbf{x})$.
- For classification into C classes, it has C units and σ_{out} is the **softmax**,

$$\text{Softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{i'=0}^{C-1} \exp(z_{i'})}, \quad i = 0, \ldots, C-1,$$

  so that $\mathbf{y}$ is a vector of C probabilities $P(Y = i \mid \mathbf{x})$ summing to 1.

Training

function TRAIN(d, f, α, B, $epochs$) **returns** the parameters θ

  $\theta \leftarrow$ random values

  repeat $epochs$ times

    for each minibatch of B pairs of d, in random order **do**

      $L \leftarrow$ the average of $\ell(y, f(x, \theta))$ over the pairs (x, y) of the minibatch

      $g \leftarrow \nabla_\theta L$, the gradient of the loss

      $\theta \leftarrow \theta - \alpha\, g$

  return θ

Same loop as for logistic regression. Only the ingredients change: the model f, its loss ℓ, and the way the gradient is computed.

Loss functions

The parameters (e.g., $\mathbf{W}_k$ and $\mathbf{b}_k$ for each layer k) of a deep network $f(\mathbf{x}; \theta)$ are learned by minimizing a loss function $\mathcal{L}(\theta)$ over a dataset $\mathbf{d} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ of input-output pairs.

The loss function is derived from the likelihood:

- For regression, assuming a Gaussian likelihood, the loss is the mean squared error $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{y}_i - f(\mathbf{x}_i; \theta)\|^2$.
- For classification, assuming a categorical likelihood, the loss is the cross-entropy $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log f_{y_i}(\mathbf{x}_i; \theta)$, the negative log-probability of the correct class.

Chain rule

The derivative of a composition of functions is the product of the derivatives along the chain. For a network with one hidden unit and a squared loss,

$$h_1 = \sigma(w_1 x + b_1), \quad \hat{y} = w_2 h_1 + b_2, \quad \ell = (y - \hat{y})^2,$$

the chain $w_1 \to h_1 \to \hat{y} \to \ell$ gives

$$\frac{\partial \ell}{\partial w_1} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_1} \frac{\partial h_1}{\partial w_1} = -2(y - \hat{y})\, w_2\, \sigma'(w_1 x + b_1)\, x.$$

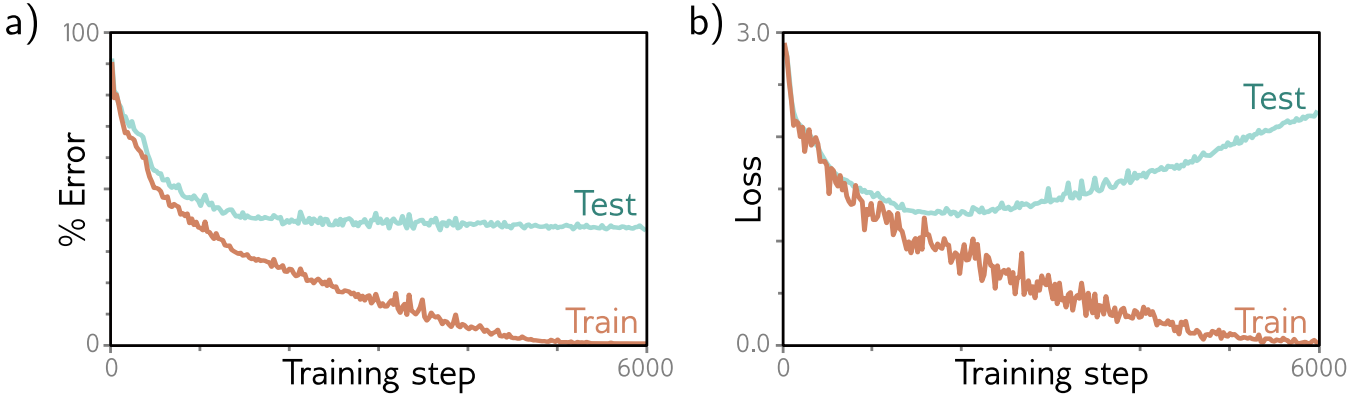
By hand, the chain rule becomes tedious and error-prone for networks with millions of parameters. **Automatic differentiation** implements it generically: the program that computes $\mathcal{L}(\theta)$ is a composition of elementary operations with known derivatives, which are multiplied along the chain. The result is the exact gradient $\nabla_\theta \mathcal{L}(\theta)$, at a small multiple of the cost of evaluating the loss once.

This applies to any such program, neural networks included. In PyTorch, it is `loss.backward()`. The training loop is unchanged.

Generalization

We want a small loss on **new** pairs drawn from $p(\mathbf{x}, y)$, not on the pairs of $\mathbf{d}$. An honest estimate comes from a **test set** $\mathbf{d}_{\text{test}}$, held out from training.

> The training loss underestimates the loss on new pairs, since θ was fitted to those very pairs.



A classifier is also judged by its **error rate**, the fraction of pairs whose predicted class $\hat{y} = \arg\max_i f_i(\mathbf{x}; \theta)$ differs from y. Gradient descent cannot minimize it: it is piecewise constant in θ, with a zero gradient almost everywhere.

A model too small for the data **underfits**: both losses stay high. A model large enough to memorize $\mathbf{d}$ **overfits**: the training loss keeps decreasing while the test loss increases, as on the previous slide.

Width, depth, learning rate, minibatch size and number of epochs are **hyperparameters**. They are chosen on a **validation set**, also held out from training. The test set is used once, at the end.

(Step-by-step code example)

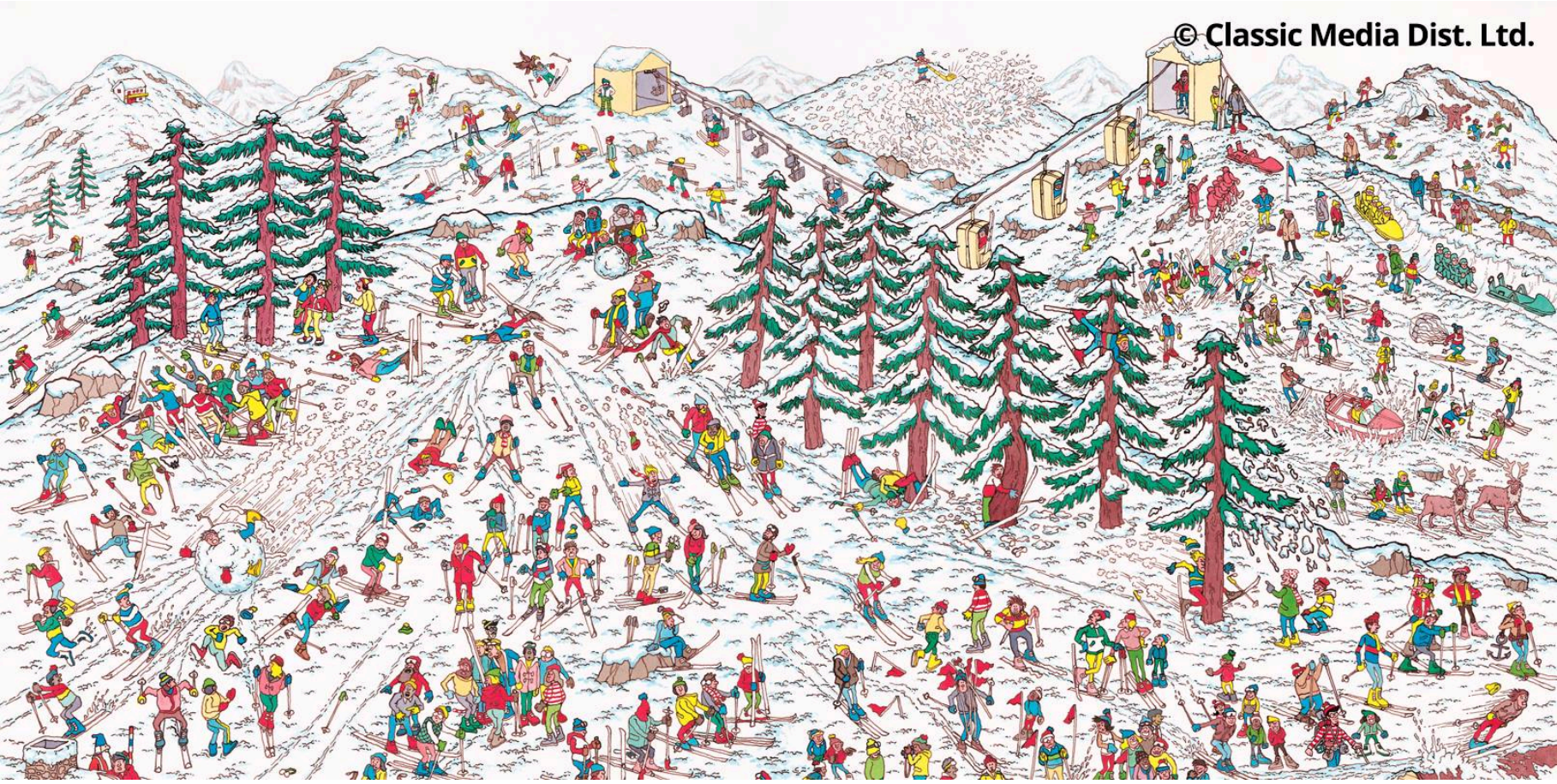
`demo/lec7/spiral.ipynb`

Convolutional networks

Images have properties that suggest the need for specialized architectures:

- They are high-dimensional inputs (e.g., 512×512 pixels = 262144 inputs).
- Nearby pixels are strongly correlated.
- The same patterns can appear at different locations in the image.
- Patterns are hierarchical: simple patterns combine to form more complex patterns.
- The interpretation of an image is stable under geometric transformations.

> The MLP architecture would fail to exploit these properties, leading to poor performance and high computational cost.



Where's Wally?

Credits: Martin Handford, Where's Wally? series.

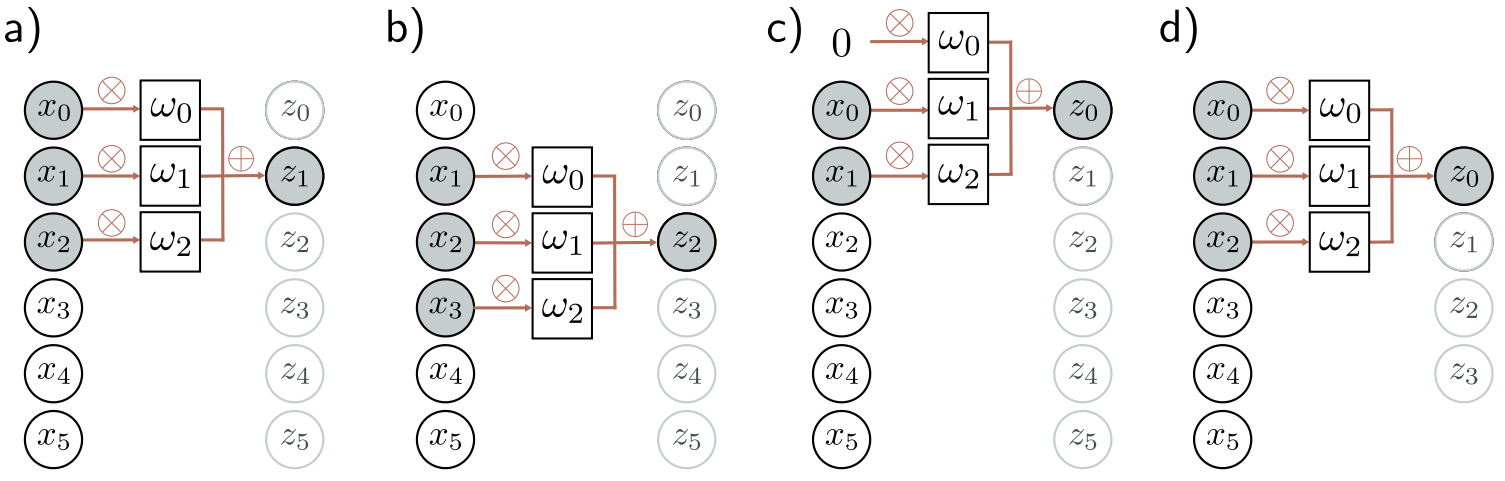
1d convolution

For the one-dimensional input $\mathbf{x} \in \mathbb{R}^W$ and the convolutional kernel $\omega \in \mathbb{R}^w$, the discrete **convolution** $\mathbf{x} \circledast \omega$ is a vector of size $W - w + 1$ such that

$$(\mathbf{x} \circledast \omega)[i] = \sum_{m=0}^{w-1} \mathbf{x}_{m+i} \omega_m.$$

Optionally, the input $\mathbf{x}$ can be padded with p zeros on each side before applying the convolution, resulting in an output of size $W - w + 1 + 2p$. Setting $p = \frac{w-1}{2}$ (for odd w) preserves the input size.

Technically, $\circledast$ denotes the cross-correlation operator. However, most machine learning libraries call it convolution.



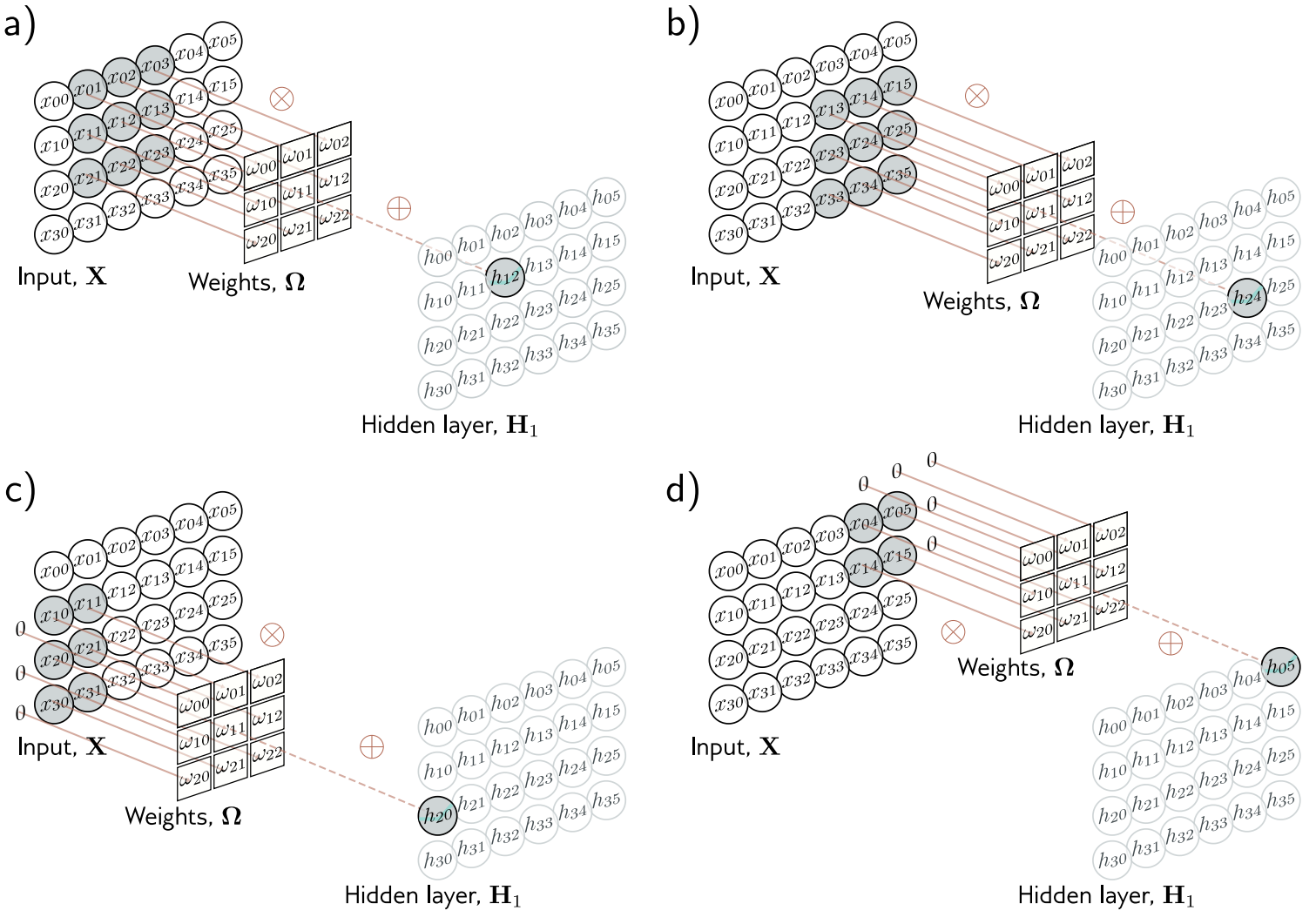
1d convolution with a kernel $\omega = (\omega_0, \omega_1, \omega_2)$ applied to the input signal $\mathbf{x}$.
The output is obtained by sliding the kernel over the input and computing the inner product at each position.

2d convolution

For the 2d input tensor $\mathbf{x} \in \mathbb{R}^{H \times W}$ and the 2d convolutional kernel $\omega \in \mathbb{R}^{h \times w}$, the discrete **convolution** $\mathbf{x} \circledast \omega$ is a matrix of size $(H - h + 1) \times (W - w + 1)$ such that

$$(\mathbf{x} \circledast \omega)[j, i] = \sum_{n=0}^{h-1} \sum_{m=0}^{w-1} \mathbf{x}_{n+j, m+i} \omega_{n,m}$$

As for 1d convolution, padding can be applied to both spatial dimensions of the input to control the output size.



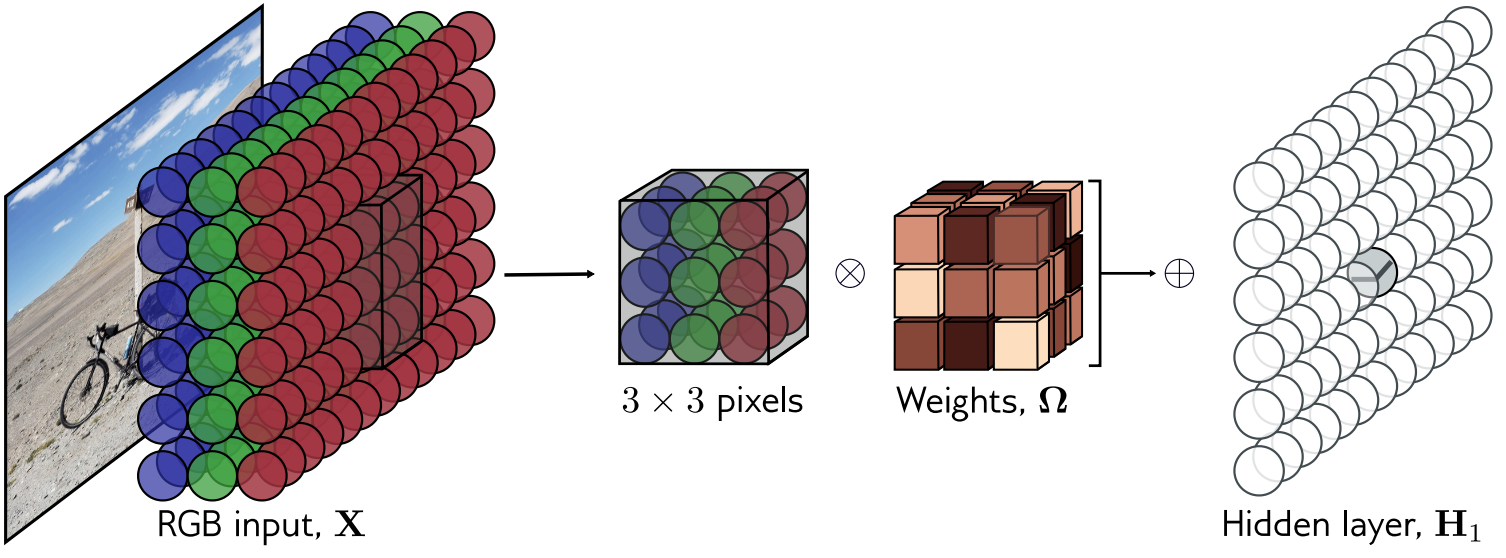
2d convolution with a kernel $\boldsymbol{\omega} \in \mathbb{R}^{3\times 3}$ applied to the input signal $\mathbf{x}$.

Channels

The 2d convolution can be extended to tensors with multiple channels.

For the 3d input tensor $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$ and the 3d convolutional kernel $\boldsymbol{\omega} \in \mathbb{R}^{C \times h \times w}$, the discrete 2d convolution $\mathbf{x} \circledast \boldsymbol{\omega}$ is a 2d tensor of size $(H - h + 1) \times (W - w + 1)$ such that

$$(\mathbf{x} \circledast \boldsymbol{\omega})[j, i] = \sum_{c=0}^{C-1} \sum_{n=0}^{h-1} \sum_{m=0}^{w-1} \mathbf{x}_{c,n+j,m+i} \boldsymbol{\omega}_{c,n,m}$$



2d convolution applied to a color image with three channels (R, G, B).

Convolutional layers

A convolutional layer is defined by a set of K kernels ω_k of size $C \times h \times w$. It applies the 2d convolution operation to the input tensor $\mathbf{x}$ of size $C \times H \times W$ to produce a set of K feature maps $\mathbf{o}_k$.

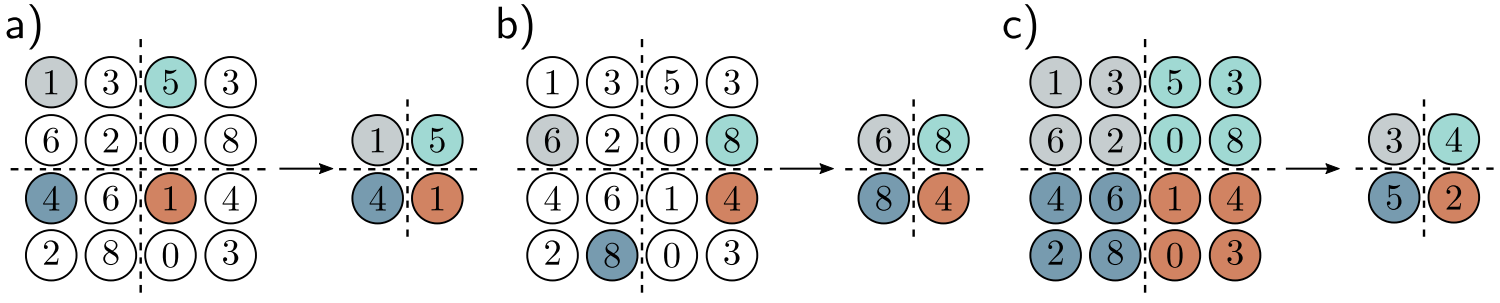
Each kernel ω_k acts as a feature detector that scans the input tensor and produces a feature map $\mathbf{o}_k$ specific to that feature.

Pooling layers

Pooling layers are used to progressively reduce the spatial size of the representation, hence capturing longer-range dependencies between features.

Considering a pooling area of size $h \times w$ and a 3D input tensor $\mathbf{x} \in \mathbb{R}^{C \times (rh) \times (sw)}$, max-pooling produces a tensor $\mathbf{o} \in \mathbb{R}^{C \times r \times s}$ such that

$$\mathbf{o}_{c,j,i} = \max_{n<h, m<w} \mathbf{x}_{c,hj+n,wi+m}.$$

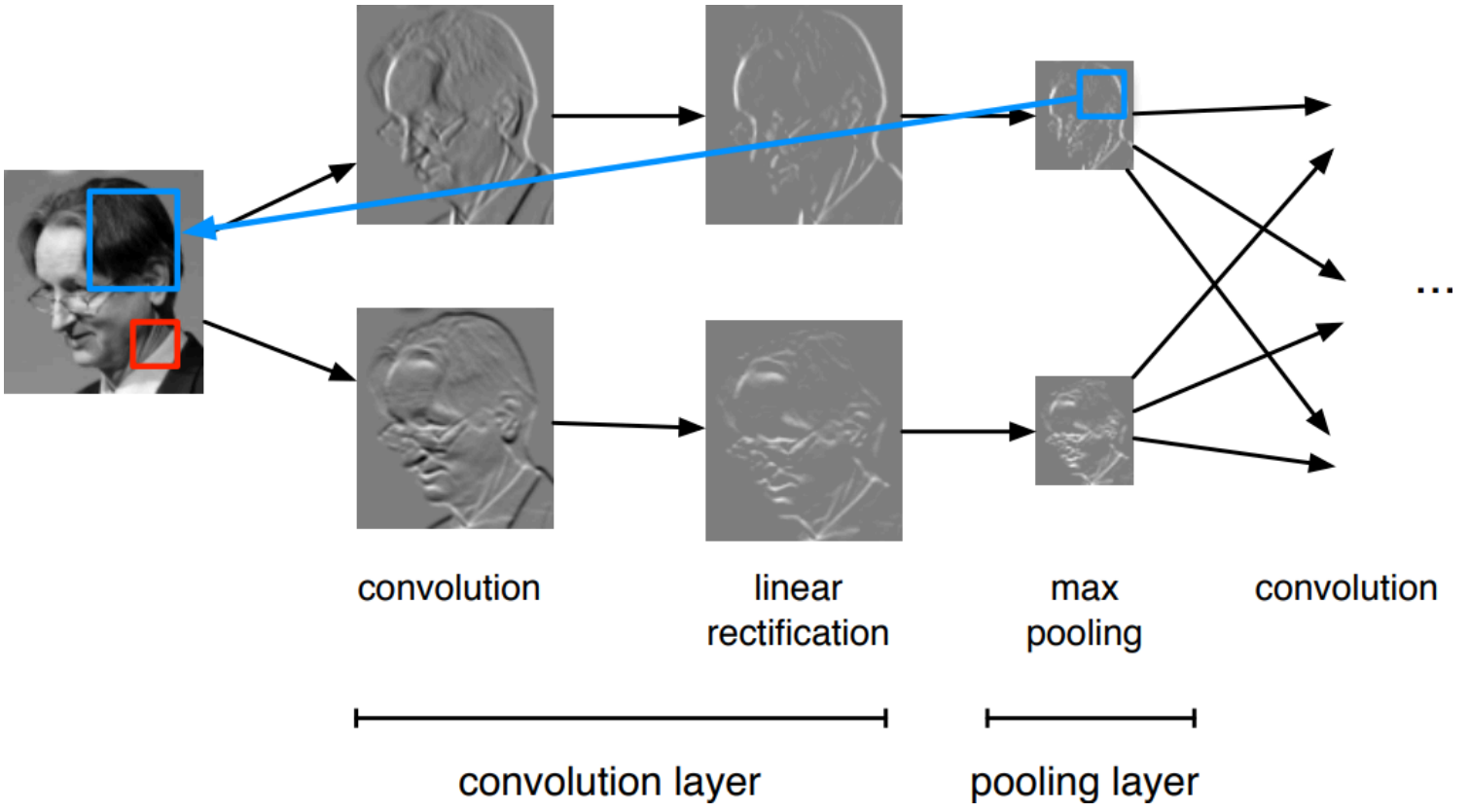


Subsampling, max-pooling and average-pooling with a pooling area of size 2×2.

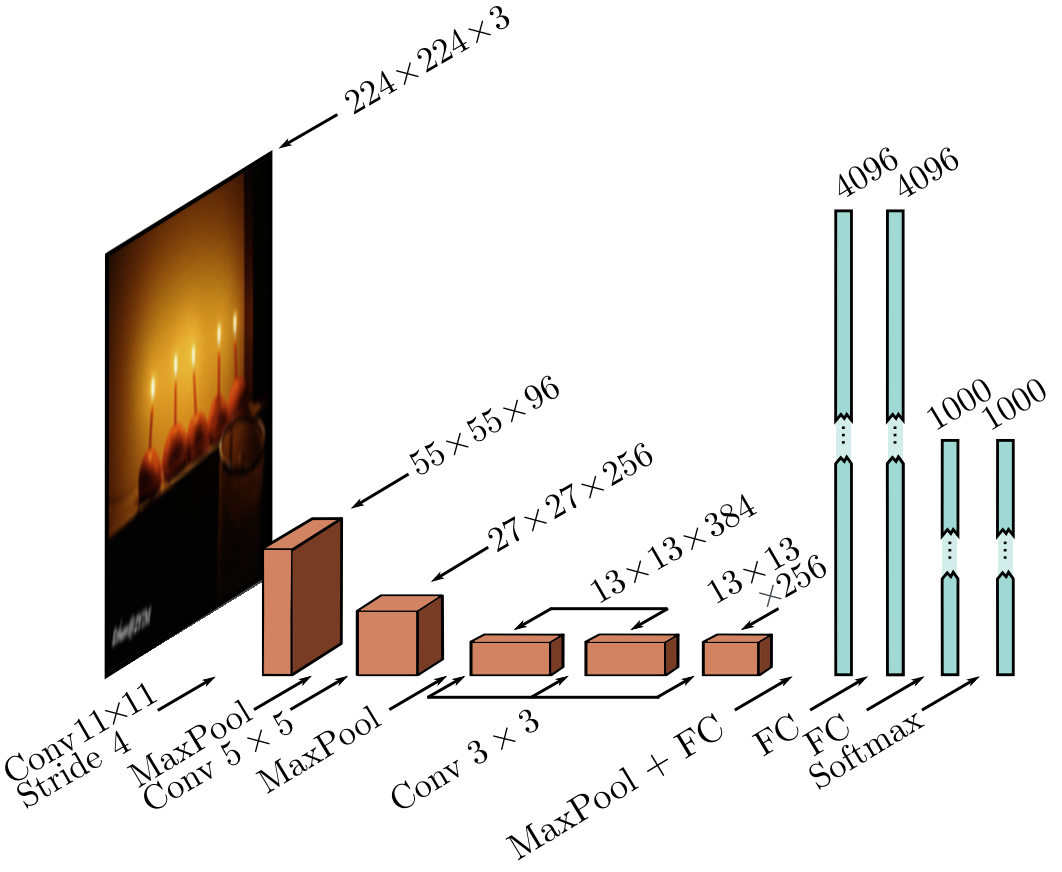
Convolutional neural networks (CNNs)

A convolutional neural network (CNN) is a deep neural network that uses convolutional layers, pooling layers, and fully connected layers to process multi-dimensional inputs such as images.

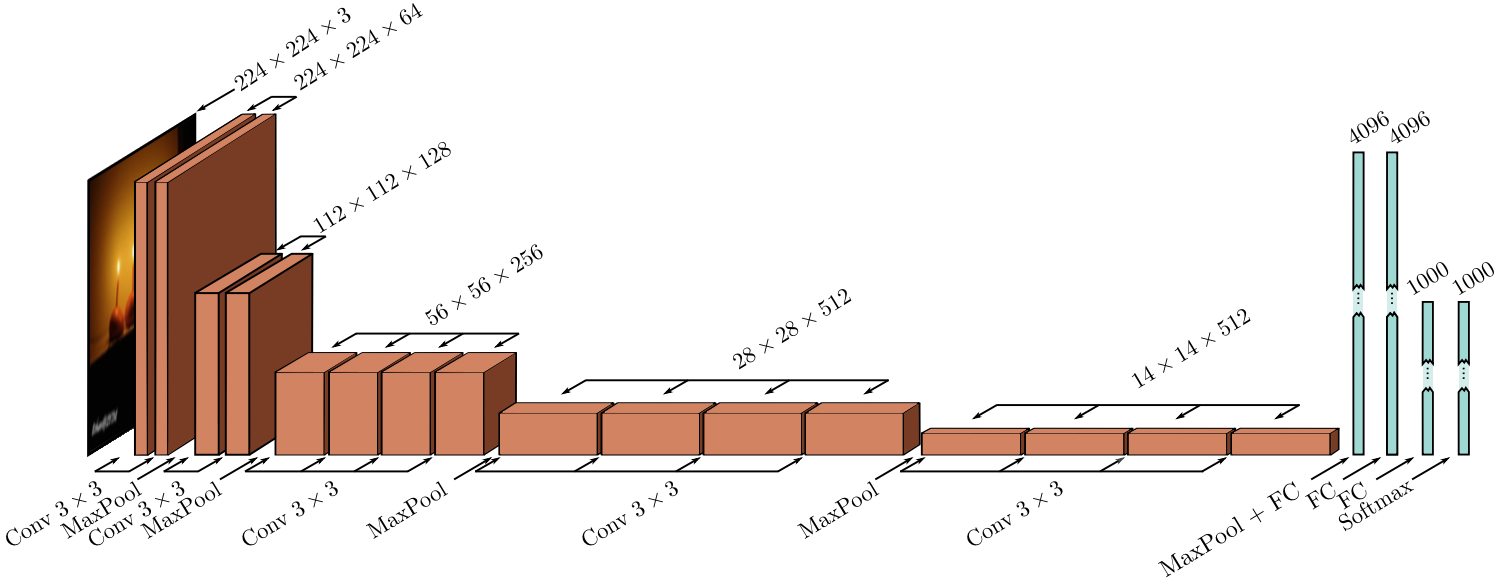
- Convolutional layers extract local features from the input while preserving spatial relationships.
- Pooling layers reduce the spatial dimensions of the feature maps, allowing the network to capture more global features.
- Fully connected layers at the end of the network combine the extracted features to make predictions.



Interleaving convolutional and pooling layers enables the network to learn hierarchical features from local to global patterns.



The AlexNet architecture for image classification (Krizhevsky et al., 2012).



The VGG architecture for image classification (Simonyan and Zisserman, 2014).

(Step-by-step code example)

`demo/lec7/convnet.ipynb`

AI beyond Pacman

Mobile ALOHA (Stanford, 2024): a human teleoperates the robot to collect the demonstrations (10x speed).

Credits: Mobile ALOHA, Fu et al., 2024.

The policies π_θ learned from those demonstrations, about 50 per task, as Pacman learns from an expert.

How AI is advancing science and medicine (Google):
images read by convolutional networks.

WeatherNext 2 (Google DeepMind, 2025): a transition model $p(\mathbf{x}_{t+1} \mid \mathbf{x}_t)$ of the atmosphere (Lecture 6), learned from decades of past weather.

Summary

- Supervised learning fits a model $p_\theta(y \mid \mathbf{x})$ to N pairs $(\mathbf{x}_i, y_i)$ by maximum likelihood, which gives the squared error for regression and the cross-entropy for classification.
- A deep network composes layers $\mathbf{h}_k = \sigma(\mathbf{W}_k^T \mathbf{h}_{k-1} + \mathbf{b}_k)$. Convolutional layers exploit the structure of images.
- Its parameters are trained by stochastic gradient descent, $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$, with gradients computed by automatic differentiation.
- Its performance is measured on pairs held out from training.
- A learning agent learns part of its program this way, e.g. a policy $a_t = \pi_\theta(e_t)$ fitted to the pairs (e_t, a_t) of an expert.

> *For the last forty years we have programmed computers; for the next forty years we will train them.*
>
> — Chris Bishop, 2020

The end.